

DOCKER MASTERY SERIES

For Vartha · DevOps Track · Product-based Company Prep

Docker, End-to-End Levels 3 · 4 · 5

Networking & Storage. Docker Compose. Production & CI/CD. One reference. Every concept. Every command. Every lab. Every interview question that actually gets asked.

LEVEL 3

Networking & Storage

Bridge, host, overlay, DNS, volumes, bind mounts, env & secrets

LEVEL 4

Docker Compose

Multi-service apps, overrides, healthchecks, dev & prod stacks

LEVEL 5

Production & CI/CD

Security, ECR, ECS, monitoring, GitHub Actions pipelines

Built for: Vartha — aspiring DevOps engineer, Hyderabad 4-6 weeks to job-ready · Do every lab

00 How to use this document

Read this first. It will save you weeks.

 **This PDF is a reference, not a textbook to read cover-to-cover.** Reading 100 pages won't make you a DevOps engineer. Doing the labs will. Your job is to alternate: read a section → run the commands → break something → debug it → come back for the next section.

The learning loop that actually works

1. **Read** the concept (don't skip the diagrams — they are the mental model).
2. **Run** every command shown. Don't just read them. Type them. Tab-complete builds muscle memory; copy-paste doesn't.
3. **Break it on purpose.** Change a flag. Use the wrong port. Forget a dependency. See what happens. *This is where real learning lives.*
4. **Lab.** Each level has hands-on labs. Do them all. No skipping the "hard" ones.
5. **Quiz.** Read the interview Q&A. Cover the answer first. Try to answer aloud as if in an interview. If you stumble, go back to the concept.

Reading the callouts

 **Tip** — a shortcut, best practice, or thing that will save you time.

 **Watch out** — common gotcha. Read carefully.

 **Pitfall** — a mistake that has cost engineers real production incidents. Burn this into memory.

 **Interview signal** — this exact thing gets asked in product-company interviews. Know it cold.

 **Lab** — hands-on exercise. Mandatory.

 **Going deeper** — optional knowledge that separates senior from junior engineers.

Pre-requisites

You should already know everything in Levels 1 and 2 of the roadmap — basic CLI (`docker run` , `ps` , `logs` , `exec`), writing Dockerfiles, multi-stage builds, image layers and caching. If you don't, stop here and complete Levels 1 & 2 first. Networking and Compose won't make sense otherwise.

 **If your laptop blocks Docker installation** — corporate IT restrictions are common. Use **Play with Docker** (labs.play-with-docker.com), **GitHub Codespaces**, or a free-tier AWS EC2 instance for all labs. Do not let the laptop be an excuse.

Time budget

- **Level 3:** ~2 weeks. 6–8 labs. Should be able to set up multi-container networking blindfolded.
- **Level 4:** ~2 weeks. Build at least 3 multi-service stacks from scratch.
- **Level 5:** ~2 weeks. End with one full pipeline: GitHub → ECR → ECS, working.

If you skip labs, you will fail the practical/scenario interview questions. Guaranteed.

TOC Table of Contents

L3 Level 3 — Networking & Storage

- 3.0 Mental model & mind map
- 3.1 Docker networking deep dive — drivers
- 3.2 Container communication & DNS
- 3.3 Port mapping & port binding
- 3.4 Storage — volumes vs bind mounts vs tmpfs
- 3.5 Volume management — create, inspect, backup
- 3.6 Environment variables & secrets (basics)
- 3.7 Command reference (Level 3)
- 3.8 Hands-on labs (6 labs)
- 3.9 Common pitfalls
- 3.10 Interview Q&A — basic, advanced, scenario

L4 Level 4 — Docker Compose

- 4.0 Mental model & mind map
- 4.1 What Compose is and when to use it
- 4.2 The docker-compose.yml structure
- 4.3 Service definition keys — every important one
- 4.4 Compose CLI commands
- 4.5 Networks & volumes in Compose
- 4.6 Healthchecks, dependencies, startup order
- 4.7 Env files, variable substitution, secrets
- 4.8 Multiple compose files — dev vs prod overrides
- 4.9 Profiles & scaling
- 4.10 Hands-on labs (6 labs)
- 4.11 Common pitfalls

4.12 Interview Q&A — basic, advanced, scenario

L5 **Level 5 — Production & CI/CD**

5.0 Mental model & mind map

5.1 Production-ready Dockerfile checklist

5.2 Security best practices

5.3 Resource management — CPU, memory, OOM

5.4 Logging, monitoring & debugging

5.5 Docker in CI/CD with GitHub Actions

5.6 Amazon ECR — private registry on AWS

5.7 AWS ECS — running containers in the cloud

5.8 Hands-on labs (6 labs)

5.9 Common pitfalls

5.10 Interview Q&A — basic, advanced, scenario

APX **Appendix — Reference & Plan**

A.1 Master command cheatsheet (1-page)

A.2 Top-30 most-asked Docker interview questions

A.3 30-day study plan to job-ready

A.4 What's beyond this PDF

Connecting containers, persisting data

How containers talk to each other, to the outside world, and how their data survives restarts.

Timeline: ~2 weeks · Labs: 6 · Q&A: 35 · Interview weight: **HIGH** — networking debug is asked in nearly every product-company round.

3.0 · Mental model

Before any commands, lock in this picture. By default, every container is a sealed box: its own network namespace, its own filesystem. You give it two superpowers explicitly: **a network to talk on** and **a place to store data that outlives the container**. That's all Level 3 is. Everything else is mechanism.

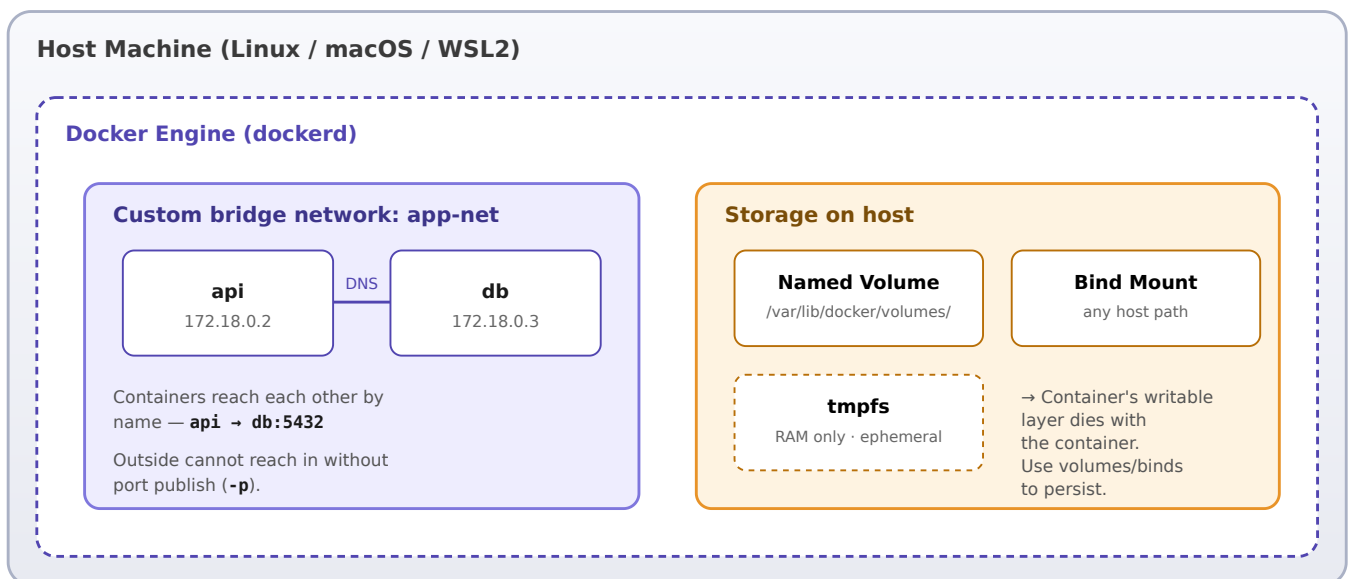


Figure 3.1 — The two superpowers: a network to talk on, a volume to remember on.

KEY TAKEAWAYS · THE QUESTIONS YOU MUST ALWAYS BE ABLE TO ANSWER

- ▶ Which network is this container on, and why?
- ▶ What's the difference between the default bridge and a user-defined bridge? (Critical interview question — answer below.)
- ▶ How does container A find container B by name? (DNS in user-defined networks.)

- ▶ If this container is deleted, does my data survive? Where does it live?
- ▶ Is this port reachable from my laptop / the internet / only inside Docker?

3.1 · Docker networking — the five drivers

A *network driver* is the engine behind a Docker network. You will use `bridge` almost always, `host` sometimes, `none` rarely, `overlay` in Swarm, and `macvlan` in very specific cases. Know all five conceptually; know bridge cold.

Driver	What it does	When to use
bridge (default)	Creates a private virtual network on the host. Each container gets its own IP inside it. Connects to outside world via NAT.	Almost every single-host use case. <i>Default for every <code>docker run</code> without <code>--network</code>.</i>
host	No network namespace isolation. The container shares the host's network directly. Container's port 80 is host's port 80.	Performance-critical apps where you don't want NAT overhead, or monitoring agents that need raw host network access. <i>Linux-only behaviour</i> — Docker Desktop on Mac/Windows fakes it.
none	Container has only a loopback interface. No external networking at all.	Highly isolated workloads — batch jobs that just read/write a volume, security-sensitive tasks.
overlay	Spans multiple Docker hosts. Containers on different machines can talk to each other as if on one network. Encrypted by default in Swarm.	Docker Swarm clusters. <i>Not for plain single-host.</i>
macvlan	Container gets a real MAC address and appears as a physical device on your LAN.	Legacy apps that expect to be on the corporate LAN directly. Rare. Almost always overkill.

The single most-asked networking question: default bridge vs user-defined bridge

Every container runs on the **default bridge network** (called `bridge`) if you don't specify one. But here is what most beginners miss:

Feature	Default bridge	User-defined bridge (docker network create)
Container-to-container DNS	NO — must use <code>--link</code> (deprecated) or IP addresses	YES — containers resolve each other by name automatically
Isolation from other networks	Weak — all default-bridge containers can see each other	Strong — only containers on this network can see each other
Dynamic attach/detach	Limited	Yes — <code>docker network connect/disconnect</code> on running containers
Recommended for production	NO	YES



Interview signal — exact phrasing that gets asked: *"Why should you never use the default bridge network for real applications?"* Answer: because it has no automatic DNS — containers can only talk to each other by IP address, which changes on every restart. User-defined bridges provide built-in DNS resolution by container name, which is what makes `docker-compose` work the way it does.

Commands — managing networks

```
# List all networks
docker network ls

# Create a user-defined bridge network
docker network create app-net
docker network create --driver bridge --subnet 172.30.0.0/16 my-custom-net

# Inspect a network (shows connected containers, IPs, subnet)
docker network inspect app-net

# Run a container on a specific network
docker run -d --name api --network app-net nginx
docker run -d --name db --network app-net postgres:16

# Attach a running container to a network (you can be on multiple nets!)
docker network connect app-net some-container
docker network disconnect app-net some-container

# Remove a network
docker network rm app-net

# Clean up unused networks
docker network prune
```



Tip: A container can be attached to multiple networks simultaneously. This is how you build "two-tier isolation" — your API container sits on both a frontend-net (reachable by load balancer) and a backend-net (reachable to DB), but the DB is only on backend-net.



Going deeper — what bridge actually is under the hood (Linux): Docker creates a Linux bridge interface (default name `docker0`) and gives each container a virtual ethernet pair (veth). One end is in the host's namespace attached to the bridge, the other is inside the container. NAT/MASQUERADE iptables rules give containers outbound internet. Knowing this answers any "how does Docker networking actually work" question — it's just Linux bridges + namespaces + iptables.

3.2 · How containers talk to each other

Container-to-container communication only works cleanly when both containers are on the same user-defined network. Then Docker's embedded DNS server (running at `127.0.0.11` inside every container) resolves container names to IPs.

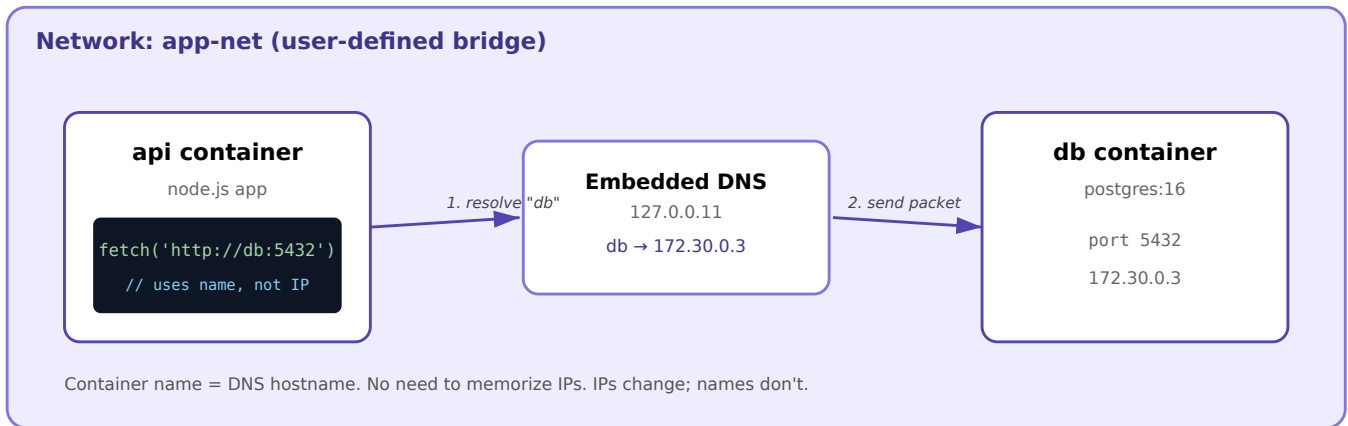


Figure 3.2 — Embedded DNS resolution on a user-defined network.

The rules of container DNS

- Container names are valid DNS hostnames on user-defined networks. `docker run --name api ...` → other containers reach it as `http://api`.
- **Network aliases** let one container respond to multiple names: `--network-alias web --network-alias frontend`.
- If a container has multiple networks, the DNS name resolves to the IP on the network the asker is on.
- Default bridge has *no* built-in DNS — names don't work, only IPs. (Hence: never use default bridge.)

Testing DNS & connectivity — the debugging commands you must memorize

```
# Confirm two containers are on the same network and can resolve each other
docker exec api ping -c 2 db
docker exec api nslookup db

# If your image is alpine-based and ping/nslookup aren't installed:
docker exec api getent hosts db
docker exec api wget -qO- http://db:5432

# Drop a debugging container into the same network
docker run --rm -it --network app-net nicolaka/netshoot
# nicolaka/netshoot has ping, curl, nslookup, dig, tcpdump, netstat — everything

# List which containers are on a network
docker network inspect app-net --format='{{range .Containers}}{{.Name}} {{.IPv4Address}}\n'
{{end}}'
```



Pitfall — "my API container can't reach Postgres": 90% of the time it's one of three things: (1) they're not on the same network, (2) you're trying to use `localhost` from inside the container (which means the container itself, not Postgres), or (3) Postgres isn't ready yet (race condition — see Compose healthchecks in Level 4).



Interview signal: *"Why is using `localhost` inside a container almost always wrong when talking to another container?"* Answer: `localhost` resolves to the container itself, not the host or other containers. Each container has its own network namespace. To reach another container, use its name (on a user-defined network) or the host's IP (if it's a host-network container).

3.3 · Port mapping — exposing containers to the outside world

By default, a container is reachable *only* from other containers on the same network. To reach it from your laptop, browser, or the public internet, you must publish a port with `-p`.

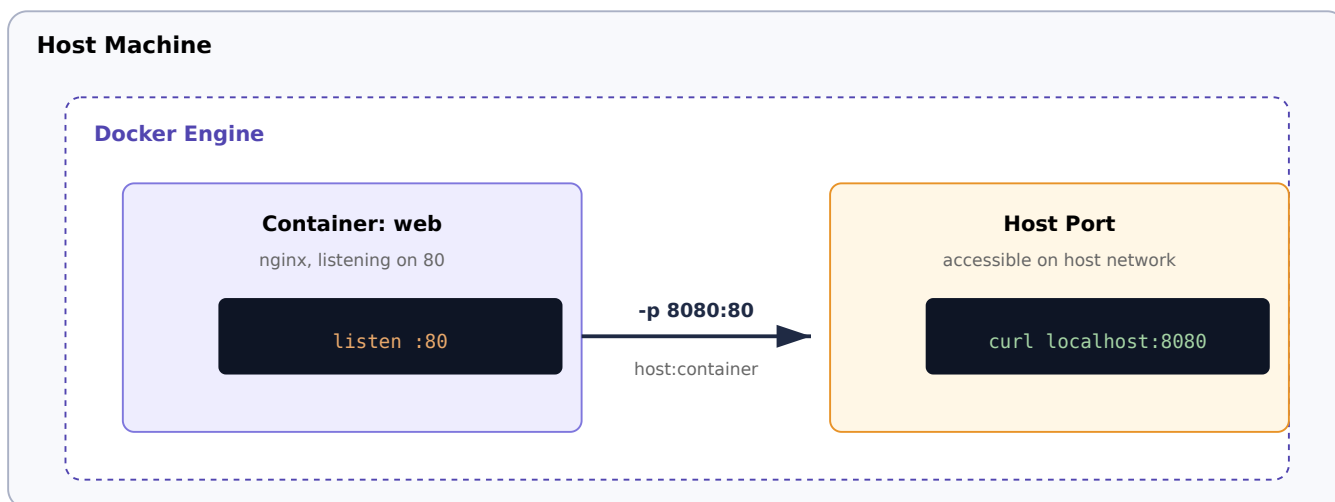


Figure 3.3 — Port publishing. The container's port 80 is exposed as host port 8080.

The `-p` flag — every variation

```
# Most common — host:container
docker run -d -p 8080:80 nginx
# localhost:8080 → container's port 80

# Multiple ports
docker run -d -p 8080:80 -p 8443:443 nginx

# Bind to specific host interface (security!)
docker run -d -p 127.0.0.1:8080:80 nginx
# Only reachable on the host itself, not from other machines

# Random host port — Docker picks one
docker run -d -p 80 nginx
docker port <container> # shows the assigned host port

# Publish ALL ports declared in EXPOSE in the Dockerfile to random host ports
docker run -d -P nginx

# UDP port
docker run -d -p 53:53/udp my-dns

# Inspect mappings on a running container
docker port my-web
```

EXPOSE vs -p — the trap question

Thing	What it does
EXPOSE 80 in Dockerfile	Documentation only. It does NOT publish the port. It signals to other tools (like <code>docker run -P</code>) and to humans which ports the container listens on.
-p 8080:80 at runtime	Actually publishes the port to the host. This is what makes the container reachable from outside Docker.



Interview signal — guaranteed asked: *"What's the difference between EXPOSE in Dockerfile and -p in docker run?"* Answer: EXPOSE is documentation/metadata; -p actually opens the port on the host. EXPOSE alone makes the container reachable to other containers on the same network (which they could do anyway by knowing the port), but does *nothing* for the host.



Pitfall — "my app is running but I can't reach it": The classic mistake. You ran `docker run -d my-app`, the container shows healthy in `docker ps`, but `curl localhost:3000` fails. You forgot `-p 3000:3000`. Container ports without publish are private. The fix is always: stop the container, restart with -p.

3.4 · Storage — volumes vs bind mounts vs tmpfs

Containers are designed to be ephemeral. The writable layer on top of an image dies when the container is removed. To persist data — databases, uploaded files, configs — you mount external storage *into* the container. There are three ways.

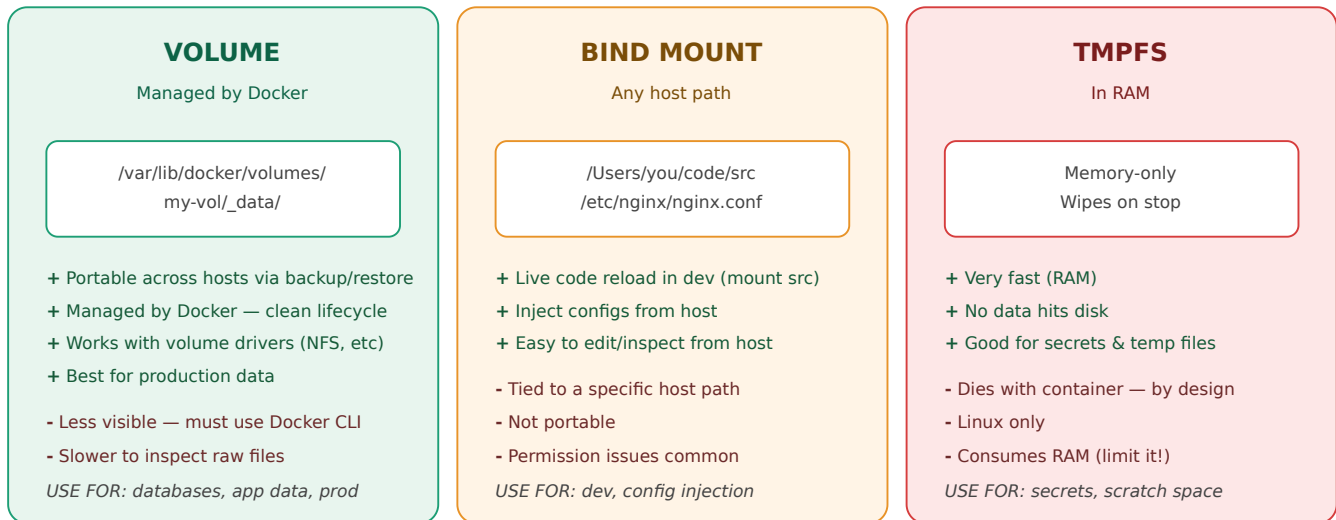


Figure 3.4 — The three storage types. Choose deliberately.

Commands — all three storage types

```
# --- VOLUMES ---
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16

# Modern syntax (more explicit, preferred in production)
docker run -d --name db \
  --mount type=volume,source=pgdata,target=/var/lib/postgresql/data \
  postgres:16

# --- BIND MOUNTS ---
# Mount your local source directory into /app for live reload during dev
docker run -d -v $(pwd)/src:/app/src --name dev-app node:20 npm run dev

# Read-only bind mount (config injection — never let container modify config)
docker run -d -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro nginx

# Modern syntax
docker run -d \
  --mount type=bind,source=$(pwd)/src,target=/app/src,readonly \
  node:20

# --- TMPFS ---
docker run -d --tmpfs /tmp:rw,size=64m nginx
docker run -d --mount type=tmpfs,destination=/tmp,tmpfs-size=64m nginx
```



Watch out — bind mount overwrites everything: if you bind-mount your host's `./src` directory to `/app` in the container, the container's existing `/app` contents (including anything you copied in via the Dockerfile) are completely hidden by your host directory. This is normal but trips up everyone the first time.



Interview signal: "*When would you choose a bind mount over a volume?*" Answer: Bind mounts shine in two cases — (1) development, where you want live code changes from your editor to appear instantly in the container, and (2) injecting host-side configuration files (like a custom `nginx.conf`) into the container. For everything else in production — especially persistent application data — named volumes are correct because they're portable, managed by Docker, and not coupled to a specific host filesystem layout.

-v vs --mount — when to use which

- `-v` is shorter, older, more permissive. It will *silently create* a missing source directory for bind mounts. Easy to typo and not notice.
- `--mount` is longer, more explicit, errors out on missing sources. Preferred in production scripts and Compose files.

3.5 · Volume management — create, inspect, backup, restore

```
# Create & list
docker volume create pgdata
docker volume ls
docker volume inspect pgdata

# Remove unused volumes (CAREFUL — may delete real data)
docker volume rm pgdata
docker volume prune

# Find where a volume lives on the host
docker volume inspect pgdata --format '{{ .Mountpoint }}'
# /var/lib/docker/volumes/pgdata/_data
```

Backing up a volume — the standard pattern

You can't `cp` a Docker volume directly (well, you can on Linux if you know the path, but the proper way works everywhere). Use a one-shot helper container:

```
# Backup pgdata volume to a tarball in current directory
docker run --rm \
  -v pgdata:/data \
  -v $(pwd):/backup \
  alpine \
  tar czf /backup/pgdata-backup-$(date +%Y%m%d).tar.gz -C /data .

# Restore from tarball
docker volume create pgdata-restored
docker run --rm \
  -v pgdata-restored:/data \
  -v $(pwd):/backup \
  alpine \
  sh -c "cd /data && tar xzf /backup/pgdata-backup-20260513.tar.gz"
```



Tip — anonymous volumes: if you write `-v /var/lib/postgresql/data` (no source name), Docker creates an *anonymous volume* with a random hash name. These accumulate fast and waste disk. Always name volumes explicitly in production: `-v pgdata:/var/lib/postgresql/data`.



Going deeper — volume drivers: beyond the default local driver, you can use NFS, Amazon EBS, Azure Files, or cloud-managed volumes via volume driver plugins. In ECS this is how persistent storage attaches to Fargate tasks (via EFS). In Kubernetes the equivalent is `PersistentVolume` — but that's CKA territory, covered in your K8s roadmap.

3.6 · Environment variables & secrets — the right way

Apps need configuration: DB URLs, API keys, log levels. The 12-factor app principle says config lives in environment variables, not in code or images. Docker supports this natively.

```
# Single variable
docker run -e NODE_ENV=production my-app

# Pass a variable from your shell into the container
export DB_PASSWORD=supersecret
docker run -e DB_PASSWORD my-app    # picks up from shell

# Load multiple from a file
docker run --env-file ./prod.env my-app

# Inspect runtime env of a running container
docker exec my-app env
docker inspect my-app --format='{{range .Config.Env}}{{println .}}{{end}}'
```

A typical `.env` file:

```
NODE_ENV=production
DB_HOST=db
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=changeme
REDIS_URL=redis://cache:6379
```



Pitfall — secrets baked into images: NEVER do `ENV DB_PASSWORD=...` in a Dockerfile, and NEVER COPY a `.env` with real secrets into an image. Image layers are forever — anyone who pulls the image can run `docker history` to see them. Even private registry images get leaked. Pass secrets at runtime.



Watch out — env vars are visible to anyone with docker inspect access: on the host machine, anyone in the docker group can see all environment variables of any running container. For real secret protection, use Docker Secrets (Swarm), AWS Secrets Manager, HashiCorp Vault, or sealed-secrets in Kubernetes. Env vars are "low-grade secrets" only.



Interview signal: "How do you handle secrets in Docker?" Answer in three layers: (1) for dev, env files outside source control; (2) for prod single-host, Docker Secrets or environment from a secrets manager fetched at boot; (3) for orchestrated prod, integrate with AWS Secrets Manager / Vault, mounted as files at runtime — never as plain env vars in image layers.

3.7 · Command reference — Level 3

Burn these into muscle memory. If you can't type them without thinking, you're not ready for the interview.

Networks

docker network ls

List all networks

docker network create app-net

Create user-defined bridge

docker network inspect app-net

Details: subnet, containers, driver

docker network rm app-net

Remove a network

docker network prune

Remove all unused networks

docker network connect app-net api

Attach running container to network

docker network disconnect app-net api

Detach from network

docker run --network app-net ...

Start container on a network

Port publishing

docker run -p 8080:80 nginx

host:container

docker run -p 127.0.0.1:8080:80 nginx

Bind to specific interface

docker run -P nginx

Publish all EXPOSEd ports randomly

docker port my-web

Show current mappings

Volumes & mounts

docker volume create pgdata

Create named volume

docker volume ls

List volumes

docker volume inspect pgdata

Show mountpoint & metadata

docker volume rm pgdata

Delete volume

docker volume prune

Delete all unused volumes

docker run -v pgdata:/data ...

Mount named volume

docker run -v \$(pwd):/app ...

Bind mount

docker run --tmpfs /tmp ...

RAM mount

Environment

docker run -e KEY=value ...

Single env var

docker run --env-file .env ...

Load env file

docker exec app env

See container's env at runtime

Debug toolkit

docker exec -it api sh

Shell into container

docker logs -f api

Follow logs

docker inspect api

Full container metadata

docker exec api ping db

Test DNS & reachability

**docker run --rm -it --network app-net
nicolaka/netshoot**

Drop in a debug container

3.8 · Hands-on labs — Level 3

Do every one. No skipping.

Lab 3.1 — User-defined network & DNS

Goal: prove that DNS works on user-defined networks and fails on the default bridge.

Step 1 — Default bridge: name resolution fails

```
docker run -d --name a alpine sleep 3600
docker run -d --name b alpine sleep 3600
docker exec a ping -c 2 b
# ping: bad address 'b' — DNS does NOT work on default bridge
docker stop a b && docker rm a b
```

Step 2 — User-defined bridge: name resolution works

```
docker network create app-net
docker run -d --name a --network app-net alpine sleep 3600
docker run -d --name b --network app-net alpine sleep 3600
docker exec a ping -c 2 b
# 64 bytes from b.app-net (172.x.x.x): seq=0 ttl=64
```

Step 3 — Inspect what's going on

```
docker network inspect app-net
docker exec a cat /etc/resolv.conf
# nameserver 127.0.0.11 ← Docker's embedded DNS
```

What you must take away

- Default bridge has no DNS. User-defined bridge does.
- Container names act as hostnames automatically — no extra config.
- `127.0.0.11` is Docker's built-in DNS resolver inside every container on a user-defined network.

Lab 3.2 — Flask + Postgres on a custom network with a persistent volume

Goal: build the milestone project from the roadmap. End state: Flask app talks to Postgres by container name; Postgres data survives container removal.

Step 1 — Set up

```
docker network create app-net
docker volume create pgdata

docker run -d --name db \
  --network app-net \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_DB=appdb \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

Step 2 — Build a tiny Flask app that connects to db

Create `app.py` :

```
from flask import Flask
import psycopg2, os
app = Flask(__name__)

@app.route("/")
def hello():
    conn = psycopg2.connect(
        host=os.environ["DB_HOST"],
        dbname=os.environ["DB_NAME"],
        user="postgres",
        password=os.environ["DB_PASSWORD"]
    )
    cur = conn.cursor()
    cur.execute("SELECT version()")
    return f"Postgres says: {cur.fetchone()[0]}"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create `Dockerfile` :

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install flask psycopg2-binary
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build & run:

```

docker build -t flask-pg-app .
docker run -d --name api \
  --network app-net \
  -p 5000:5000 \
  -e DB_HOST=db \
  -e DB_NAME=appdb \
  -e DB_PASSWORD=secret \
  flask-pg-app

curl http://localhost:5000
# Postgres says: PostgreSQL 16.x ...

```

Step 3 — Prove data persists

```

docker exec -it db psql -U postgres -d appdb -c "CREATE TABLE t(x int); INSERT INTO t VALUES (42);"
docker stop db && docker rm db
docker run -d --name db --network app-net \
  -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=appdb \
  -v pgdata:/var/lib/postgresql/data postgres:16
docker exec -it db psql -U postgres -d appdb -c "SELECT * FROM t;"
# x | 42 ← survived container deletion!

```

Lab 3.3 — Bind mount for live-reload dev

Goal: change source code on the host and see it reflected in the running container without rebuilding.

```

mkdir live-demo && cd live-demo
echo '<h1>Hello v1</h1>' > index.html

docker run -d --name web \
  -p 8080:80 \
  -v $(pwd):/usr/share/nginx/html:ro \
  nginx

curl localhost:8080
# <h1>Hello v1</h1>

echo '<h1>Hello v2 (live!)</h1>' > index.html
curl localhost:8080
# <h1>Hello v2 (live!)</h1> ← no rebuild!

```

Lab 3.4 — Volume backup & restore

Goal: backup a volume to a tarball and restore it to a new volume.

```
# Assume pgdata from Lab 3.2 has data
docker run --rm \
  -v pgdata:/data \
  -v $(pwd):/backup \
  alpine \
  tar czf /backup/pgdata-backup.tar.gz -C /data .

ls -lh pgdata-backup.tar.gz

# Restore to a brand new volume
docker volume create pgdata-restored
docker run --rm \
  -v pgdata-restored:/data \
  -v $(pwd):/backup \
  alpine \
  sh -c "cd /data && tar xzf /backup/pgdata-backup.tar.gz"

# Start new Postgres pointing at restored volume
docker run -d --name db-restored --network app-net \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata-restored:/var/lib/postgresql/data \
  postgres:16

docker exec -it db-restored psql -U postgres -d appdb -c "SELECT * FROM t;"
# 42 ← the data is back
```

Lab 3.5 — Multi-network isolation

Goal: put a container on two networks so it acts as a bridge between isolated tiers.

```
docker network create frontend-net
docker network create backend-net

docker run -d --name db --network backend-net postgres:16
docker run -d --name lb --network frontend-net -p 8080:80 nginx

# API container sits on BOTH
docker run -d --name api --network frontend-net nginx
docker network connect backend-net api

# Verify
docker exec api ping -c 1 db    # works — api is on backend-net
docker exec lb ping -c 1 db     # FAILS — lb is not on backend-net (the point!)
```

This is the standard 3-tier isolation pattern. Your DB is never directly reachable from the load balancer tier.

Lab 3.6 — Debug a broken connection (scenario practice)

Goal: practice the muscle of debugging when containers can't talk.

```
# Intentionally broken setup – different networks
docker network create net-a
docker network create net-b
docker run -d --name api --network net-a alpine sleep 3600
docker run -d --name db --network net-b alpine sleep 3600

docker exec api ping db
# bad address 'db' ← because they're on different networks
```

Now **your job is to debug it** using only these commands:

1. `docker inspect api` → what network is api on?
2. `docker network inspect net-a` → who else is on this network?
3. Conclusion: db is on a different network.
4. Fix: `docker network connect net-a db`
5. Re-test: `docker exec api ping db` → works.

This exact flow is what interviewers test. They give you a broken setup; you debug aloud.

3.9 · Common pitfalls — networking & storage



Using default bridge for real apps. No DNS, weak isolation. Always `docker network create` first.



Using localhost inside a container to mean another container. `localhost` is the container itself. Use the other container's name on a user-defined network.



Forgetting `-p` and wondering why the browser can't reach the app. A "running" container doesn't mean "exposed to host." Always check `docker ps` — if the `PORTS` column is empty, you forgot to publish.



Bind-mounting over a directory that the image already populated. The bind hides the image's contents. Symptom: an empty `node_modules` after mounting your source. Solution: either install deps in a sub-mount, use a named volume for `node_modules`, or do `npm install` inside the container after start.



Accumulating anonymous volumes. Every restart of certain images (Postgres, MySQL) without an explicit volume creates a new anonymous one. `docker volume ls` fills with garbage hashes. Fix: always name the volume; periodically run `docker volume prune` (carefully).



Mounting host paths with wrong permissions. Mac/Linux container processes often run as root inside but write files owned by host-root, locking you out. Use `--user $(id -u):$(id -g)` or specify a user in the Dockerfile.



Putting secrets in ENV in Dockerfile. Image history exposes them forever. Pass at runtime only.



Trusting EXPOSE to publish a port. It doesn't. EXPOSE is metadata only. You need `-p`.



Using host network on Docker Desktop (Mac/Windows). It doesn't work the way you expect — Docker Desktop runs containers in a Linux VM. `--network host` means "host of the VM", not your macOS host. Linux only.



Race conditions — app starts before DB is ready. Just because db is on the network doesn't mean Postgres is accepting connections. Covered in Level 4 with healthchecks; for plain Docker, your app needs a connection-retry loop on boot.

3.10 · Interview Q&A — Level 3 (networking & storage)

35 questions. Cover the answer before reading it. Say the answer aloud. Then check.

BASIC · 15 QUESTIONS

Q. What are the default network drivers in Docker?

A. Bridge (default for new containers), host (no isolation, shares host network), none (no network at all), overlay (multi-host, for Swarm), and macvlan (container gets a MAC address on physical network).

Q. What network do containers join if you don't specify one?

A. The default `bridge` network. It exists out of the box and is automatically attached to every container started without `--network`.

Q. How do you create a user-defined bridge network?

A. `docker network create my-net`. Optionally with flags like `--driver bridge --subnet 172.30.0.0/16`.

Q. What is the main difference between default bridge and user-defined bridge?

A. User-defined bridges have automatic DNS — containers resolve each other by name. Default bridge has no DNS; you'd need IPs (which change on restart). Always use user-defined bridges.

Q. How does container A reach container B by name?

A. Both must be on the same user-defined network. Docker's embedded DNS server (at `127.0.0.11` inside each container) resolves container names to current IPs.

Q. What does `-p 8080:80` mean?

A. Publish host port 8080 to container port 80. Traffic to `localhost:8080` on the host is forwarded to port 80 inside the container.

Q. Difference between `EXPOSE` in Dockerfile and `-p` at runtime?

A. `EXPOSE` is documentation/metadata — it does not actually publish anything. `-p` is what makes the port reachable from the host. `EXPOSE` is consumed by tools like `-P` (publish all exposed ports randomly).

Q. What is a Docker volume?

A. A persistent storage location managed by Docker, stored under `/var/lib/docker/volumes/` by default. Survives container removal. Used for databases and persistent app data.

Q. Difference between a volume and a bind mount?

A. A volume is managed by Docker — you reference it by name, Docker decides where on disk. A bind mount is a direct mapping from a specific host path. Volumes are portable; bind mounts are tied to a host's filesystem.

Q. When would you use a bind mount?

A. Development (live code reload) and config injection (mount a host config file into the container, often read-only).

Q. What is tmpfs and when do you use it?

A. A RAM-only mount. Data is lost when the container stops. Useful for sensitive scratch data (secrets temporarily decrypted, generated tokens) or fast-access temporary files. Linux only.

Q. How do you pass environment variables to a container?

A. `-e KEY=value` for one, `--env-file path` for many. Inside Dockerfile use `ENV` for non-sensitive defaults only.

Q. Why is putting secrets in `ENV` in a Dockerfile bad?

A. Image layers are immutable history. Anyone with the image can run `docker history` and see them. Pass secrets at runtime, never bake them in.

Q. How do you list, inspect, and remove networks?

A. `docker network ls`, `docker network inspect <name>`, `docker network rm <name>`. Bulk cleanup: `docker network prune`.

Q. How do you list, inspect, and remove volumes?

A. `docker volume ls`, `docker volume inspect <name>`, `docker volume rm <name>`. Bulk: `docker volume prune`.

ADVANCED · 10 QUESTIONS

Q. Can a container be on multiple networks simultaneously?

A. Yes. Attach with `docker network connect` after start, or with multiple `--network` definitions in Compose. Standard use case: a multi-network bridge container that bridges frontend and backend tiers but keeps them isolated from each other.

Q. What is the difference between `-v` and `--mount`?

A. Both achieve the same things but `--mount` is more explicit (key=value syntax) and safer — it errors out on misconfiguration like missing source paths, whereas `-v` silently creates them. Prefer `--mount` in scripts/production.

Q. How does Docker bridge networking actually work under the hood?

A. Docker creates a Linux bridge interface (e.g., `docker0`) on the host. Each container gets a virtual ethernet pair (`veth`); one end is in the container's network namespace, the other is attached to the bridge. Iptables NAT/MASQUERADE rules give containers outbound internet. It's all Linux networking primitives — Docker just orchestrates them.

Q. How do you backup a Docker volume?

A. Run a helper container that mounts the volume read-only and a host directory, then `tar czf` the volume contents into the host directory: `docker run --rm -v pgdata:/data -v $(pwd):/backup alpine tar czf /backup/x.tar.gz -C /data .`

Q. What's an anonymous volume and why is it dangerous?

A. A volume without an explicit name, created when you mount with `-v /path/inside/container` (no source). Docker assigns a random hash. They accumulate, are hard to identify, and easy to lose data in. Always name volumes.

Q. What happens when you set `--network host` on a container?

A. The container shares the host's network namespace entirely. No NAT, no port translation — the container's port 80 is the host's port 80. Best for performance-sensitive workloads, but breaks port isolation (two containers can't both bind 80). Note: behaves differently on Docker Desktop (Mac/Windows), which runs containers in a Linux VM.

Q. How do you make a container only accessible to `localhost` on the host?

A. Bind the publish to 127.0.0.1: `docker run -p 127.0.0.1:8080:80 nginx`. Other machines on the network cannot reach it.

Q. What are network aliases and when are they useful?

A. `--network-alias <name>` lets a container respond to additional DNS names on a network. Useful for blue/green deployments (alias `api` can swing between two containers) or service abstraction (DB container responds to both `db` and `postgres`).

Q. What is the role of `127.0.0.11` inside a container?

A. It's Docker's embedded DNS resolver. Every container on a user-defined network has its `/etc/resolv.conf` pointed at 127.0.0.11. It resolves container names, network aliases, and forwards external queries upstream.

Q. Why might `docker exec` show different env vars than `docker inspect`?

A. `docker inspect` shows the env passed at *container start*. The process inside might have modified its own environment, or the container might have a startup script that sets additional vars. `docker exec env` reflects the live shell's environment as a subprocess sees it.

Q. Your API container is on `app-net`. Postgres is on `app-net`. From the API, `ping db` works but `curl db:5432` times out. What's wrong?

A. Postgres isn't ready yet, OR the Postgres container hasn't bound to `0.0.0.0` (it might be listening only on loopback inside the container), OR you have a firewall/security tool blocking port 5432 inside the container. Debug: `docker exec db netstat -tln` to see what's actually listening; `docker logs db` to confirm Postgres started without errors.

Q. Your team deployed a new version of the app and now `localhost:3000` returns connection refused. `docker ps` shows the container as Up. Walk me through the debug.

A. (1) Check `docker ps` PORTS column — is the port actually published? Maybe the new compose file dropped `-p`. (2) `docker logs <container>` — is the app actually listening, or did it crash silently? (3) `docker exec <container> ss -tlnp` — is the app bound to `0.0.0.0` or `127.0.0.1`? Apps bound to `127.0.0.1` inside the container are unreachable from the host even with `-p`.

Q. You changed application code on the host and the container isn't reflecting the change. The Dockerfile does `COPY . /app`. What's the fix for dev?

A. `COPY` bakes code into the image — it freezes at build time. For dev, add a bind mount of `$(pwd):/app` at runtime, OR use a tool like nodemon/fresh inside the container. In production, rebuild the image; never bind-mount source in prod.

Q. After many deploys you notice `docker volume ls` has hundreds of anonymous volumes. Disk is full. How do you safely clean up?

A. (1) `docker volume ls -qf dangling=true` lists unused volumes. (2) Confirm none belong to currently stopped-but-needed containers. (3) `docker volume prune` removes them. For prevention, name every volume explicitly and audit volume usage in deploy scripts.

Q. You need to expose a service to other containers on the host but NOT to the outside world. How?

A. Put the service on a user-defined network and skip `-p` entirely. Other containers on the same network reach it by name and port. The outside world (including the host) cannot, because no port is published.

Q. A junior engineer ran `docker run -v $(pwd):/app node:20` and now their `node_modules` from a previous Docker build is gone. Why?

A. Bind mount overwrites the container's `/app` directory with the host's empty (or empty-of-`node_modules`) version. The image's `/app/node_modules` is hidden. Fix: either run `npm install` after start, or use a named volume for `/app/node_modules` so it doesn't get overwritten by the bind.

Q. Your monitoring agent needs to see all network traffic on the host. Which network mode?

A. `--network host`. The agent shares the host's network namespace and sees everything directly. Trade-off: no port isolation. (Or use `--cap-add NET_ADMIN` with a more nuanced setup.)

Q. You need to give a developer access to inspect a production volume's contents without giving them shell on the host. How?

A. Run a temporary read-only container that mounts the volume: `docker run --rm -it -v pgdata:/data:ro alpine sh`. Developer gets file-level access via the container; host shell is untouched. Better still — give them a sandboxed file manager image, not shell.

Q. A container starts, runs for 2 seconds, then exits. Logs show nothing. How do you debug?

A. (1) `docker logs <id>` — capture last gasp output. (2) `docker inspect <id> --format='{{.State.ExitCode}}'` — exit code tells you about signals/errors. (3) Override the entrypoint to drop into a shell: `docker run -it --entrypoint sh my-image`. (4) Then manually run what your app would've run, watch where it fails. Common cause: missing env var, wrong file path, immediate crash on bad config.

Q. Postgres data must persist *and* be backed up nightly. Design the storage strategy.

A. (1) Run Postgres with a named volume: `-v pgdata:/var/lib/postgresql/data`. (2) For backup, prefer logical backups via `pg_dump` over volume tar — Postgres data files require a consistent snapshot. (3) Run a cron'd backup container: `docker exec db pg_dump -U postgres appdb | gzip > /backup/dump-$(date +%F).sql.gz`. (4) Push the backup off-host (S3, etc). Volume-tar backups are OK only if Postgres is stopped during the tar — not acceptable for a live service.

Multi-service apps as code

Declarative orchestration for the single-host world. The bridge to Kubernetes thinking.

Timeline: ~2 weeks · Labs: 6 · Q&A: 35 · Interview weight: **VERY HIGH** — Compose YAML reading is asked in every Docker round.

4.0 · Mental model

Compose is a YAML-driven wrapper that runs many `docker run` commands for you, declared in one file. Every flag you painfully typed in Level 3 — `--network`, `-v`, `-e`, `-p`, `--depends-on` — becomes a YAML key. One `docker compose up` brings the whole stack to life. **If you can run it as separate docker run commands, you can write it as a Compose file. Compose is not magic — it's automation over what you already know.**

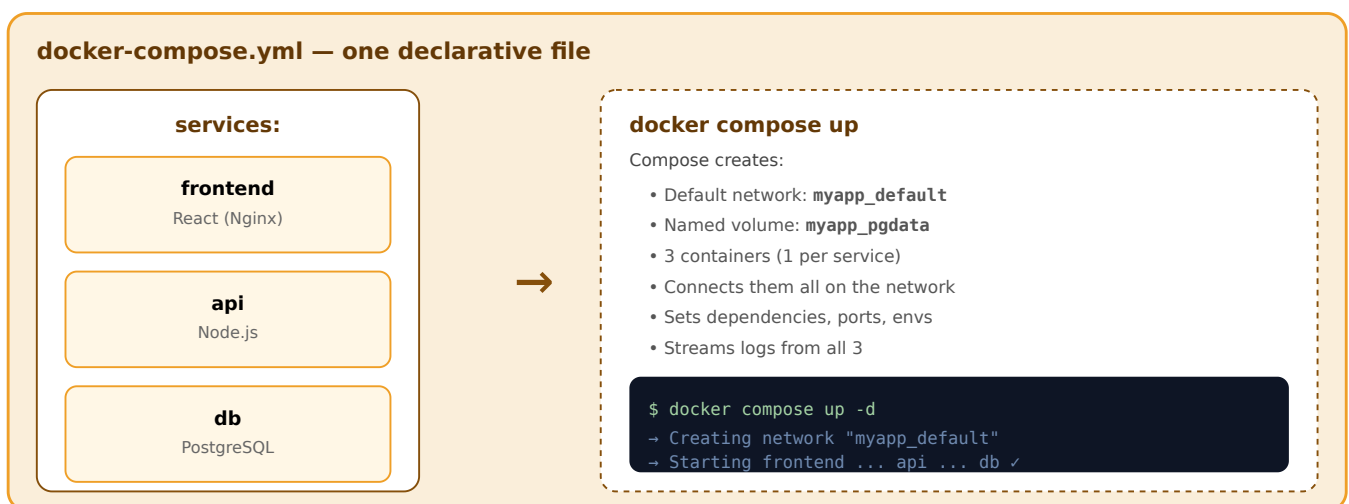


Figure 4.1 — Compose: declarative YAML to a fully-wired stack with one command.

WHY COMPOSE MATTERS FOR YOUR CAREER

- ▶ Every multi-service local-dev setup at every company uses Compose.
- ▶ Reading and writing `docker-compose.yml` is a Day-1 skill — interviewers will show you one and ask "what does this do?"
- ▶ Mental model maps cleanly to Kubernetes Deployments/Services later. Compose-fluent → Kubernetes is way easier.
- ▶ For interview prep, you should be able to write a 3-service compose file (frontend, API, DB) from memory in 5 minutes.

4.1 · What Compose is and when to use it

What it is

Docker Compose is a tool to define and run multi-container Docker applications using a single YAML file (`docker-compose.yml` or just `compose.yml` in v2+). It uses the regular Docker Engine API under the hood — there's no separate runtime.

 **Compose v1 vs v2 — know the difference:** Compose v1 was a separate Python tool, invoked as `docker-compose` (with a hyphen). Compose v2 is a Go plugin built into Docker, invoked as `docker compose` (no hyphen). v1 is deprecated. **Use `docker compose` (space) in 2026.**

When Compose is the right tool

Use Compose	Don't use Compose
Local development environments — DB + cache + API + frontend	Multi-host production at scale (use Kubernetes / ECS / Swarm)
CI test environments — spin up a stack, run tests, tear down	Auto-scaling beyond one host
Demo/staging on a single VM	Apps requiring rolling updates with zero-downtime guarantees
Self-hosted single-server apps (a home Plex, a small SaaS MVP)	When the team is already on Kubernetes — don't fragment tooling

 **Interview signal — exact question:** *"When would you NOT use Docker Compose?"* Strong answer: when you need multi-host, when you need auto-scaling, when you need zero-downtime rolling updates, or when the rest of your infra is on Kubernetes — Compose is fundamentally single-host. The right escalation path is Compose → Swarm → Kubernetes / ECS.

4.2 · The `docker-compose.yml` structure

The top-level keys you'll encounter:

```
name: myapp                                # project name (sets the prefix for nets/vols)

services:                                  # containers to run
  frontend:
    image: nginx:alpine
  api:
    build: ./api
  db:
    image: postgres:16

networks:                                   # custom networks (optional – default network is created if
omitted)
  frontend-net:
  backend-net:

volumes:                                    # named volumes
  pgdata:
  redis-data:

configs:                                    # config files mounted into services (v3.3+)

secrets:                                    # swarm-mode secrets, mounted as files
```

Full 3-tier example — read this carefully, you will be asked about a file

like this

```
name: myapp

services:

  frontend:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./frontend/dist:/usr/share/nginx/html:ro
    depends_on:
      - api
    networks: [frontend-net]
    restart: unless-stopped

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    environment:
      DB_HOST: db
      DB_NAME: appdb
      DB_USER: postgres
      DB_PASSWORD: ${DB_PASSWORD}      # pulled from .env at compose-up time
      REDIS_URL: redis://cache:6379
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    networks: [frontend-net, backend-net]
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d appdb"]
      interval: 5s
      timeout: 3s
      retries: 5
    networks: [backend-net]

  cache:
    image: redis:7-alpine
    networks: [backend-net]
    restart: unless-stopped

networks:
  frontend-net:
  backend-net:
```

```
volumes:  
  pgdata:
```

What this gives you with one `docker compose up -d`:

- 4 containers (frontend, api, db, cache), each isolated to the right networks (frontend can't see db directly — only api can).
- 1 persistent named volume for Postgres data.
- api waits for db to be *healthy*, not just started, before launching — critical to avoid race conditions.
- The `${DB_PASSWORD}` variable comes from your `.env` file (next to `docker-compose.yml`), so secrets aren't in the YAML.

4.3 · Service definition keys — every important one

Key	What it does	Example
<code>image</code>	Pull from a registry	<code>postgres:16</code>
<code>build</code>	Build from a Dockerfile locally	<code>build: ./api</code> or with <code>context / dockerfile</code> sub-keys
<code>container_name</code>	Force a specific name (loses ability to scale)	<code>container_name: my-db</code>
<code>command</code>	Override the image's CMD	<code>command: ["python", "-m", "http.server", "8000"]</code>
<code>entrypoint</code>	Override the image's ENTRYPOINT	<code>entrypoint: /entrypoint.sh</code>
<code>environment</code>	Set env vars (list or map)	<code>environment: { DB_HOST: db }</code>
<code>env_file</code>	Load env from file(s)	<code>env_file: [./.env, ./env.local]</code>
<code>ports</code>	Publish to host (<code>host:container</code>)	<code>ports: ["8080:80"]</code>
<code>expose</code>	Make port visible to linked services (no host publish)	<code>expose: ["5432"]</code>
<code>volumes</code>	Mount volumes / bind mounts	<code>- pgdata:/var/lib/postgresql/data</code> or <code>- ./src:/app:ro</code>
<code>networks</code>	Networks this service attaches to	<code>networks: [frontend-net, backend-net]</code>
<code>depends_on</code>	Start order & readiness gating	See section 4.6
<code>healthcheck</code>	How Docker checks the service is alive	See section 4.6
<code>restart</code>	Restart policy	<code>no</code> , <code>always</code> , <code>on-failure</code> , <code>unless-stopped</code>
<code>user</code>	Run as specific UID/GID	<code>user: "1000:1000"</code>
<code>working_dir</code>	Working directory in container	<code>working_dir: /app</code>
<code>profiles</code>	Group services for selective startup	<code>profiles: ["debug", "monitoring"]</code>
<code>deploy</code>	Resource limits, replicas (some keys only honored in Swarm)	<code>deploy: { resources: { limits: { memory: 512m } } }</code>
<code>logging</code>	Log driver and options	<code>logging: { driver: json-file, options: { max-size: 10m } }</code>
<code>tty</code> / <code>stdin_open</code>	Allocate TTY / keep STDIN open (for interactive)	<code>tty: true</code>



About container_name: setting it explicitly is convenient for one-off services but blocks scaling — Docker can't run two containers with the same name. For services you might scale (`docker compose up --scale api=3`), omit `container_name` and let Compose auto-generate names like `myapp-api-1`, `myapp-api-2`.



Going deeper — deploy nuances: in plain Compose (not Swarm), only some deploy keys work — `resources` (limits/reservations) and `restart_policy` are honored. `replicas`, `placement`, `update_config`, `rollback_config` are Swarm-only. Knowing this lets you read complex Compose files without getting confused.

4.4 · Compose CLI commands

Lifecycle

```
# Bring up the whole stack (creates networks, volumes, containers)
docker compose up
docker compose up -d                # detached (no log streaming)
docker compose up --build           # rebuild images before starting
docker compose up --force-recreate  # recreate containers even if config didn't change

# Stop and remove everything created by up
docker compose down
docker compose down -v              # ALSO delete named volumes (DANGEROUS – wipes DB data)
docker compose down --rmi all       # ALSO delete images

# Start/stop without removing
docker compose start
docker compose stop
docker compose restart api
```

Inspection

```
docker compose ps                  # list containers in this stack
docker compose logs                # aggregated logs from all services
docker compose logs -f api         # follow one service
docker compose logs --tail=100 db
docker compose top                 # show running processes per service
docker compose config              # print the fully resolved compose file (with env vars
filled in)
docker compose images              # list images used
```

Interaction

```
docker compose exec api sh         # shell into a running service
docker compose exec db psql -U postgres
docker compose run --rm api npm test # one-off command in a NEW container
docker compose run --rm --no-deps api echo "isolated run"
```



Difference between exec and run: exec attaches to an already-running container. run creates a brand-new container from the service config, runs the given command, then exits. Use exec for live debugging; use run --rm for tests, migrations, or one-shot tasks.

Building

```
docker compose build               # build all services that have a `build:` key
docker compose build --no-cache api
docker compose pull                # pull all images from registry
docker compose push                # push built images (requires image: with registry name)
```


Scaling (single-host)

```
docker compose up -d --scale api=3
docker compose ps
# myapp-api-1, myapp-api-2, myapp-api-3 – all on same network
```



Pitfall — docker compose down -v in production: the -v flag deletes named volumes. Run it on a production stack and your database data is GONE. Many production incidents trace to this single character. *Memorize the danger.*

4.5 · Networks & volumes in Compose

Default behavior — what Compose auto-creates

If you have *no* `networks:` top-level block, Compose creates one network named `<project>_default` and puts every service on it. Services reach each other by service name. **This is why a minimal Compose file "just works" with multi-service DNS.**

Explicit networks for tier isolation

```
services:
  lb:
    networks: [frontend-net]
  api:
    networks: [frontend-net, backend-net]
  db:
    networks: [backend-net]

networks:
  frontend-net:
  backend-net:
```

Now `lb` cannot reach `db` directly. Only `api` can. Tier isolation in 4 lines of YAML.

External networks & volumes

```
networks:
  shared-net:
    external: true                # created outside this compose file

volumes:
  legacy-data:
    external: true                # pre-existing volume
```

Useful when multiple Compose projects share a network or volume (e.g., a shared monitoring stack).

Volume definitions

```
services:
  db:
    volumes:
      - pgdata:/var/lib/postgresql/data      # named volume
      - ./backups:/backups                  # bind mount (host-relative)
      - /etc/timezone:/etc/timezone:ro      # bind mount, read-only
      - type: tmpfs                          # tmpfs via long syntax
        target: /tmp
        tmpfs:
          size: 67108864

volumes:
  pgdata:                                  # declare named volume at top level
  pgdata-named:
    driver: local
    driver_opts:
      type: nfs
      o: addr=10.0.0.5,nolock
      device: ":/exports/pgdata"
```

4.6 · Healthchecks, dependencies, startup order

Compose starts services in dependency order, but **"started" ≠ "ready"**. A Postgres container starts in 1 second; it's ready to accept connections after 3-5. If your API connects in second 2, it crashes. This is the most common bug in Compose stacks.

The wrong way — naive depends_on

```
services:
  api:
    depends_on:
      - db          # waits for db CONTAINER to start, not for Postgres to be ready
  db:
    image: postgres:16
```

The right way — depends_on + healthcheck

```
services:
  api:
    depends_on:
      db:
        condition: service_healthy    # waits for db's healthcheck to pass
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d appdb"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s                # grace period before retries start counting
```

The conditions you can use

Condition	Meaning
<code>service_started</code>	Container has started. <i>Default — usually not enough.</i>
<code>service_healthy</code>	Container's healthcheck has passed. What you almost always want.
<code>service_completed_successfully</code>	Container exited with code 0. For one-shot init containers (e.g., DB migrations before the app starts).

Common healthcheck patterns

```
# Postgres
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  retries: 5

# MySQL
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 5s
  retries: 10

# Redis
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 5s

# Generic HTTP service
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 30s  # app needs 30s to warm up; don't fail it before that
```



Interview signal — guaranteed asked: *"How do you ensure your API container only starts after Postgres is actually ready to accept connections?"* Answer in two parts: (1) use `depends_on: condition: service_healthy`, and (2) define a meaningful healthcheck on the Postgres service using `pg_isready`. Plain `depends_on` only waits for container start, not application readiness. **Mentioning `start_period` for slow-booting apps gets bonus points.**



Going deeper — why your app should still retry: healthchecks aren't bulletproof. In production, networks blip, DBs restart, dependencies fail. Your application code should always have a connection-retry loop with backoff, regardless of whether Compose health-gates the startup. Compose healthchecks are insurance, not a guarantee.

4.7 · Env files, variable substitution, secrets

The `.env` file — Compose's default

A file named exactly `.env` in the same directory as `docker-compose.yml` is loaded automatically. Variables in it are available for substitution in the YAML.

```
# .env
DB_PASSWORD=changeme-in-prod
POSTGRES_VERSION=16
APP_PORT=8080
```

```
# docker-compose.yml
services:
  db:
    image: postgres:${POSTGRES_VERSION}
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
  api:
    ports:
      - "${APP_PORT}:3000"
```

Defaults & fallbacks:

```
${VAR:-default}      # use VAR or "default" if unset/empty
${VAR-default}       # use VAR or "default" if unset (empty is ok)
${VAR:?error msg}     # error if VAR is unset/empty – useful for required secrets
```

Service-level env_file

```
services:
  api:
    env_file:
      - ./env
      - ./env.api
      - ./env.local    # later files override earlier ones
```



Pitfall — committing .env with real secrets: add .env to .gitignore from day one.

Commit a .env.example with dummy values instead. Real engineers have leaked production secrets this way.

Compose secrets (Swarm/v3.1+)

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    file: ./db_password.txt    # or `external: true` in Swarm
```

Secrets get mounted as files under `/run/secrets/`. The app reads from the file path, not an env var. More secure (less exposed to `docker inspect` and process listings).

4.8 · Multiple compose files — dev vs prod overrides

You will have different configs for dev (live bind mount of source, debug logging, exposed DB ports for direct access) and prod (no bind mount, prod logging, DB not directly exposed). Compose's override mechanism handles this elegantly.

The default override file

If `docker-compose.override.yml` exists, Compose automatically merges it on top of `docker-compose.yml`. Often used for dev-only tweaks.

`docker-compose.yml` (base — production-like):

```
services:
  api:
    build: ./api
    image: myapp/api:latest
    environment:
      LOG_LEVEL: info
    restart: always
```

`docker-compose.override.yml` (dev-only — auto-loaded):

```
services:
  api:
    volumes:
      - ./api/src:/app/src          # live reload
    environment:
      LOG_LEVEL: debug
    ports:
      - "5432:5432"                 # expose DB to local tools (NEVER in prod)
    command: ["npm", "run", "dev"]
```

Now `docker compose up` in dev gives the merged behaviour. In production, you skip the override:

```
docker compose -f docker-compose.yml up -d
# uses base only — no override merged
```

Explicit multi-file pattern

```
# Files:
#   docker-compose.yml      - common base
#   docker-compose.dev.yml  - dev overrides
#   docker-compose.prod.yml - prod overrides

# DEV
docker compose -f docker-compose.yml -f docker-compose.dev.yml up

# PROD
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# Print the merged result to verify
docker compose -f docker-compose.yml -f docker-compose.prod.yml config
```

Files are merged in order. Later files override earlier ones for scalar values; lists are concatenated for some keys (volumes, ports). When in doubt, run `docker compose config` to see the final result.

Profiles — another way to slice

Profiles let you mark services as "only start in this scenario."

```
services:
  app: { image: my-app }
  debug-shell:
    image: alpine
    profiles: ["debug"]           # only starts with --profile debug
    command: sleep infinity
  prometheus:
    image: prom/prometheus
    profiles: ["monitoring"]

# Usage
docker compose up -d                # starts just `app`
docker compose --profile monitoring up -d  # starts app + prometheus
docker compose --profile debug --profile monitoring up -d
```



Tip: profiles are cleaner than commenting out services. Use them for optional pieces (a debug toolbox, a load-test container, a metrics exporter).

4.9 · Scaling services in Compose

```
docker compose up -d --scale api=3
docker compose ps
# myapp-api-1    Up 5 seconds  9000/tcp
# myapp-api-2    Up 5 seconds  9000/tcp
# myapp-api-3    Up 5 seconds  9000/tcp
```

Important constraints when scaling:

- **No `container_name`** on the scaled service (forces a single fixed name — incompatible with multiple instances).

- **No host port collisions** — you can't have 3 instances all binding host port 8080. Either remove the `ports:` entry (only internal traffic) or use a range like `"3000-3009:3000"` for auto-allocation.
- **DNS round-robin** — when another container resolves the service name (`api`), Docker returns multiple IPs, and the client load-balances naively. For real load balancing, put a reverse proxy (Nginx, Traefik) in front.

Pattern: Nginx reverse proxy load-balancing multiple API instances

```
services:
  api:
    build: ./api
    expose: ["3000"]           # internal only, no host port
    deploy:
      replicas: 3             # Swarm-mode; for plain compose, use --scale

  nginx:
    image: nginx:alpine
    ports: ["80:80"]
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - api
```

`nginx.conf` upstream:

```
upstream api_backend {
    server api:3000;           # Docker DNS returns all api instances
}

server {
    listen 80;
    location / { proxy_pass http://api_backend; }
}
```


4.10 · Hands-on labs — Level 4

Lab 4.1 — Simplest possible 2-service stack

Goal: write your first Compose file from scratch. Run it. Tear it down.

```
# File: compose.yml
services:
  web:
    image: nginx:alpine
    ports: ["8080:80"]
  redis:
    image: redis:7-alpine

# Run
docker compose up -d
docker compose ps
docker compose logs
docker compose exec web sh -c "wget -qO- http://redis:6379 || echo cant"
docker compose exec redis redis-cli ping

# Tear down
docker compose down
```

Observe: no networks block in YAML, yet web can resolve redis. That's the default network at work.

Lab 4.2 — Full 3-tier stack (Frontend + API + DB + Cache)

Goal: build the milestone project from the roadmap. Mirror real production architecture.

Project layout:

```
myapp/
├── compose.yml
├── .env
├── frontend/
│   ├── Dockerfile
│   └── index.html
├── api/
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
└── nginx.conf
```

Build out a real version with Node.js API, a Postgres DB, Redis cache, and Nginx serving a static frontend. Use environment variables for DB credentials. Use healthchecks. Use two networks for tier isolation.

Acceptance criteria — your stack passes the lab when:

- `docker compose up -d` brings up all 4 services and they all become healthy.
- Frontend container cannot reach the DB directly (proves network isolation).
- API only starts after DB is healthy (proves condition: `service_healthy` works).
- After `docker compose down` and `up` again, DB data is still there (proves named volume).
- Stack works on a fresh clone with just `cp .env.example .env` and `docker compose up`.



This is the lab interviewers will recreate. Many product-company Docker rounds are essentially "build me this stack live, narrate as you go." Doing this lab to mastery is the highest-leverage prep you can do for Level 4.

Lab 4.3 — Dev override for live code reload

Goal: extend Lab 4.2 with a dev override that bind-mounts source code, runs the API with nodemon, and exposes the DB port for local pgAdmin access. All without touching the base compose file.

```
# compose.override.yml (auto-loaded in dev)
services:
  api:
    volumes:
      - ./api:/app
      - /app/node_modules      # anonymous volume – prevents bind from hiding node_modules
    command: ["npm", "run", "dev"]
    environment:
      LOG_LEVEL: debug
  db:
    ports: ["5432:5432"]      # expose for local tools
```

Test by editing a route in `server.js` and watching the container restart automatically.

Lab 4.4 — Production override

Goal: create a `compose.prod.yml` that disables bind mounts, sets resource limits, enables proper logging.

```
# compose.prod.yml
services:
  api:
    image: myregistry/api:v1.2.3  # pinned tag, no build
    build: !reset null             # explicitly disable build (compose v2.20+)
    restart: always
    deploy:
      resources:
        limits: { cpus: "1.0", memory: 512m }
    logging:
      driver: json-file
      options: { max-size: "10m", max-file: "3" }
  db:
    restart: always
    deploy:
      resources:
        limits: { memory: 1g }
    # db port NOT exposed – only api can reach it

# Usage
docker compose -f compose.yml -f compose.prod.yml up -d
```

Lab 4.5 — Healthchecks & conditional start

Goal: prove that `service_healthy` actually delays API startup.

Use `postgres:16` with a slow init script. Add an init container that runs DB migrations and must complete successfully before API starts:

```
services:
  db:
    image: postgres:16
    environment: { POSTGRES_PASSWORD: pass }
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 3s
      retries: 10

  migrate:
    image: my-migrator:latest
    depends_on:
      db: { condition: service_healthy }
    command: ["/run-migrations.sh"]

  api:
    image: my-api:latest
    depends_on:
      migrate: { condition: service_completed_successfully }
      db: { condition: service_healthy }
```

Watch the order: in another terminal, run `docker compose logs -f` while the stack comes up. You'll see `db` → `migrate` → `api`, with explicit waits.

Lab 4.6 — Scale a stateless service with reverse proxy

Goal: scale API to 3 replicas behind Nginx. Watch round-robin load balancing.

```
docker compose up -d --scale api=3
docker compose exec nginx nslookup api
# Returns 3 IPs — one per replica
for i in $(seq 1 9); do curl -s localhost/ ; done
# Should hit different containers — check logs to confirm
docker compose logs api | grep "Handled request"
```

4.11 · Common pitfalls — Docker Compose



Naive depends_on without healthchecks. Compose only waits for container start, not application readiness. Race conditions on every up.



docker compose down -v in production. Deletes named volumes — your DB is gone. Memorize: -v in down = nuke from orbit.



Hardcoding secrets in YAML. Use .env + .gitignore, or secrets manager. Never commit real credentials.



Using container_name on services you'll scale. Scaling fails because Docker can't have two containers with the same name. Remove container_name for scalable services.



Forgetting :ro on config bind mounts. If the container writes to a config file, you've now permanently changed your host config. Always mount configs read-only.



Bind-mounting source code over /app and losing node_modules. Solution: anonymous volume on /app/node_modules shields the image's deps from the host bind.



Confusing docker compose run and docker compose exec. run spawns a fresh container (and may leave it around without --rm). exec attaches to one already running. Use run --rm for one-shots, exec for debugging live containers.



Compose v1 syntax (docker-compose) habits. In v2, it's docker compose (with a space). Some flags differ. Check docker compose --help if you're unsure.



Volume mount paths confusion in WSL2 / Windows. Host-relative paths like ./data work, but absolute Windows paths (C:\Users\...) need careful escaping. Stick to relative paths in compose files for portability.



Putting build: and image: both for a service. Compose builds the image, then tags it with the value of image:. Useful for pushing built images to a registry, but confuses people who think they're mutually exclusive.

4.12 · Interview Q&A — Level 4 (Docker Compose)

BASIC · 15 QUESTIONS

Q. What problem does Docker Compose solve?

A. Running multi-container applications declaratively. Instead of typing many `docker run` commands with flags for network, volumes, env, ports — you declare everything in one YAML file and run `docker compose up`.

Q. What's the difference between `docker-compose (v1)` and `docker compose (v2)`?

A. v1 is the standalone Python tool (with hyphen). v2 is a Go plugin built into Docker (no hyphen, just a space). v1 is deprecated. Always use v2 in 2025+.

Q. What top-level keys can you have in a Compose file?

A. name, services, networks, volumes, configs, secrets. services is the only one strictly required.

Q. Difference between `image:` and `build:`?

A. `image:` pulls a prebuilt image from a registry. `build:` builds an image from a local Dockerfile. You can use both — Compose builds and then tags the result with the `image` value, which is useful for pushing to a registry.

Q. Difference between `ports:` and `expose:` in Compose?

A. `ports:` publishes to the host (host:container). `expose:` only declares which ports are reachable from other containers on the same network — no host binding. Same as Dockerfile EXPOSE + container-to-container.

Q. How do services in the same Compose file reach each other?

A. By service name as DNS hostname. Compose automatically creates a default network (or uses the ones you defined) and puts all services on it. `api` can reach `db` as `http://db:5432`.

Q. What does `depends_on` do?

A. Controls startup order. By default it only waits for the dependency container to start. With `condition: service_healthy` it waits until the dependency's healthcheck passes — which is what you usually want.

Q. How do you define a healthcheck?

A. Under the service: a `healthcheck:` block with `test` (a command), `interval`, `timeout`, `retries`, and optionally `start_period`. Example: `test: ["CMD-SHELL", "pg_isready -U postgres"]`.

Q. What does `docker compose down -v` do?

A. Stops and removes containers, networks, AND named volumes. The `-v` is dangerous — it deletes your data. Don't run on production.

Q. How do you pass an environment variable to a service?

A. `environment`: as a list or map, or `env_file`: pointing to a file. Variables can use `${VAR}` substitution from a `.env` file in the project dir.

Q. What does `${DB_PASSWORD:?error message}` do?

A. It fails Compose with the error message if `DB_PASSWORD` is unset or empty. Useful for required secrets — fails loudly instead of starting with an empty password.

Q. How do you run a one-off command in a Compose service?

A. `docker compose run --rm api npm test`. Creates a fresh container from the service config, runs the command, removes the container.

Q. How do you scale a service?

A. `docker compose up -d --scale api=3`. Requires no `container_name` and no host port collision (don't bind a fixed host port on a scaled service).

Q. What's the default network Compose creates?

A. A bridge network named `<project>_default`, where `project` is the directory name or the value of `name`:. All services join it unless you specify `networks`: on the service.

Q. How does Compose decide the project name?

A. In order: the top-level `name`: key, the `COMPOSE_PROJECT_NAME` env var, or the directory name. Project name prefixes all resources (networks, volumes, container names).

ADVANCED · 10 QUESTIONS

Q. What's the difference between `condition: service_started`, `service_healthy`, and `service_completed_successfully`?

A. `service_started`: container is up (default). `service_healthy`: healthcheck has passed. `service_completed_successfully`: container exited with code 0 — used for one-shot init containers (like migrations) that must succeed before the next service starts.

Q. How do you handle dev vs prod with Compose?

A. Multiple files merged via `-f`: a base `compose.yml` for shared config, plus `compose.dev.yml` or `compose.prod.yml` as overrides. Compose merges later files over earlier ones. Or use `compose.override.yml` for an automatically-loaded dev tweak.

Q. How does Compose handle merging when you pass multiple files?

A. Scalar values: later overrides earlier (e.g., image, command). Lists: depending on the key — for ports and volumes, lists are concatenated; for others, lists are replaced. Always run `docker compose config` to inspect the merged result.

Q. When would you use Compose profiles?

A. When some services should only start in specific contexts — a debug shell, a load tester, a metrics exporter. Mark them with profiles: `["debug"]` and they only start with `--profile debug`. Cleaner than commenting out services.

Q. How do Compose secrets differ from environment?

A. Secrets are mounted as files inside `/run/secrets/<name>`, not exposed as environment variables. Apps must read from the file path. More secure: not visible to `docker inspect` or process listings. Most useful in Swarm or when using `POSTGRES_PASSWORD_FILE`-style image conventions.

Q. Can you use Compose with multiple Docker hosts?

A. Plain Compose: no — it's single-host. The historical way was `docker swarm` with `docker stack deploy -c compose.yml` (which uses a Compose file but is Swarm-orchestrated). For real multi-host, the modern answer is Kubernetes (or ECS, which has its own task definition format).

Q. What's an "external" network or volume in Compose?

A. One that was created outside this Compose project (by hand or another project). You reference it with `external: true`. Compose won't create or delete it — it just attaches to it. Used for sharing data/networks across projects.

Q. What's the difference between Compose's default network and a named user-defined network?

A. The default is auto-created and named `<project>_default`; all services join it. Named networks (declared under top-level `networks:`) let you put different services on different networks for isolation. Otherwise they're identical bridge networks.

Q. When does `docker compose up` recreate a container vs reuse it?

A. It recreates if the service config has changed (image, env, volumes, etc) since the last up. `--force-recreate` recreates regardless. `--no-recreate` reuses even if config changed (rarely useful).

Q. What's the practical difference between `image + build together` vs `just build`?

A. If you only set `build:`, Compose builds an image and gives it an auto-generated name. If you set both, Compose builds and tags the result with your `image:` value — useful for then doing `docker compose push` to a registry, or having a stable image name for production deploys.

Q. Your stack works locally but in CI the API starts before Postgres is ready and crashes. Fix it.

A. Add a healthcheck to the db service using `pg_isready`, and set the api's `depends_on` to `condition: service_healthy`. Also: app code should still retry on connection failure as a defense-in-depth measure — healthchecks aren't bulletproof.

Q. A colleague ran `docker compose down -v` on staging and wiped the test database. How do you prevent this culturally and technically?

A. (1) Technical: enable regular volume backups (cron'd `pg_dump` to S3). (2) Cultural: code review for any script using `down -v` on shared environments. (3) Add a Makefile target like `make nuke-data` that requires explicit confirmation rather than relying on raw compose commands. (4) Document clearly in the project README: `down -v` is local-only.

Q. You need a clean DB on every CI test run, but persistent DB in dev. Same compose file. How?

A. Base `compose.yml` uses a named volume. Add `compose.ci.yml` override that uses `tmpfs` for the DB data directory — instant teardown, no persistence. CI runs with `-f compose.yml -f compose.ci.yml`; dev runs default.

Q. A service is occasionally OOM-killed in prod. How do you investigate and prevent it?

A. (1) Confirm: `docker inspect <container> --format '{{.State.OOMKilled}}'` returns true after restart. (2) Set memory limit via `deploy.resources.limits.memory` AND a reservation. (3) Add monitoring on `docker stats` or use cAdvisor. (4) Profile the app for memory leaks. The container memory limit doesn't fix a leak — it just makes the symptom visible.

Q. Your compose stack has Frontend + API + DB. Frontend should be reachable from the internet; DB should be reachable only from API. How do you structure networks?

A. Two networks: `frontend-net` and `backend-net`. Frontend on `frontend-net` with ports: `["80:80"]`. API on both networks (acts as bridge). DB only on `backend-net` with **no** ports: `entry`. Result: DB is unreachable from the outside, even from the frontend; only API can talk to it.

Q. You have to run database migrations once per deploy, before the app starts. Design the Compose setup.

A. Three services: `db` (with healthcheck), `migrate` (one-shot, depends on db being healthy, exits with 0 on success), `api` (depends on `migrate` with `condition: service_completed_successfully`, and db healthy). On `compose up`, the order is enforced; on subsequent deploys, the `migrate` container re-runs idempotently.

Q. You want a developer to spin up your stack with one command, no manual setup. What's missing if your compose file requires editing `.env` first?

A. Provide `.env.example` committed to git with placeholder values (and a copy-instruction in the README). Add to Makefile: `make dev` that copies `.env.example` to `.env` if missing, then runs `docker compose up`. Compose itself supports `${VAR:-default}` for non-critical defaults.

Q. Your stack has 10 services. `docker compose up` is slow and you can't tell what's failing. How do you debug faster?

A. (1) `docker compose up -d` then `docker compose ps` to see status. (2) `docker compose logs -tail=50 -f <service>` per service. (3) Use profiles to bring up just the subset you're working on. (4) `docker compose config` to verify the merged YAML is what you expect. (5) Run problem services manually with `docker compose run --rm <service> sh` to interactively debug.

Q. Your frontend container needs to reach the API at `http://api:3000` but the API has multiple replicas. How does this work and where can it go wrong?

A. Docker DNS returns multiple A records for the service name, and the client picks one. Most HTTP clients reuse connections, so a single long-lived client might stick to one replica (no load balancing). The fix in serious setups is a reverse proxy (Nginx, Traefik) that proxies to `api:3000` — the proxy does proper load balancing because it makes a fresh DNS lookup per request or pools.

Q. Show me a Compose snippet that demonstrates you understand secrets, healthchecks, networks, and dev overrides — in one config.

A. Reference your Lab 4.2 result and walk through it: explicit `frontend-net/backend-net`, `condition: service_healthy` on the DB, `${DB_PASSWORD}` from `.env` (with `.env` in `.gitignore`), and a `compose.override.yml` that adds bind mounts and exposes the DB port for local tools. *The interviewer wants to see you using all four mechanisms together, not separately.*

From "it works on my laptop" to "it runs in production"

Security, resource limits, observability, GitHub Actions pipelines, Amazon ECR, AWS ECS. The skills that get product-company offers.

Timeline: ~2 weeks · Labs: 6 · Q&A: 35 · Interview weight: **CRITICAL** — every DevOps round tests pipeline knowledge and security awareness.

5.0 · Mental model

Production isn't "Compose with bigger machines." It's a different mode of thinking. Three concerns dominate: **security** (an exploited container can compromise the host or other tenants), **resource sanity** (one rogue container can't be allowed to take down the node), and **repeatability** (every deploy must be identical, auditable, and reversible — which is what CI/CD gives you).

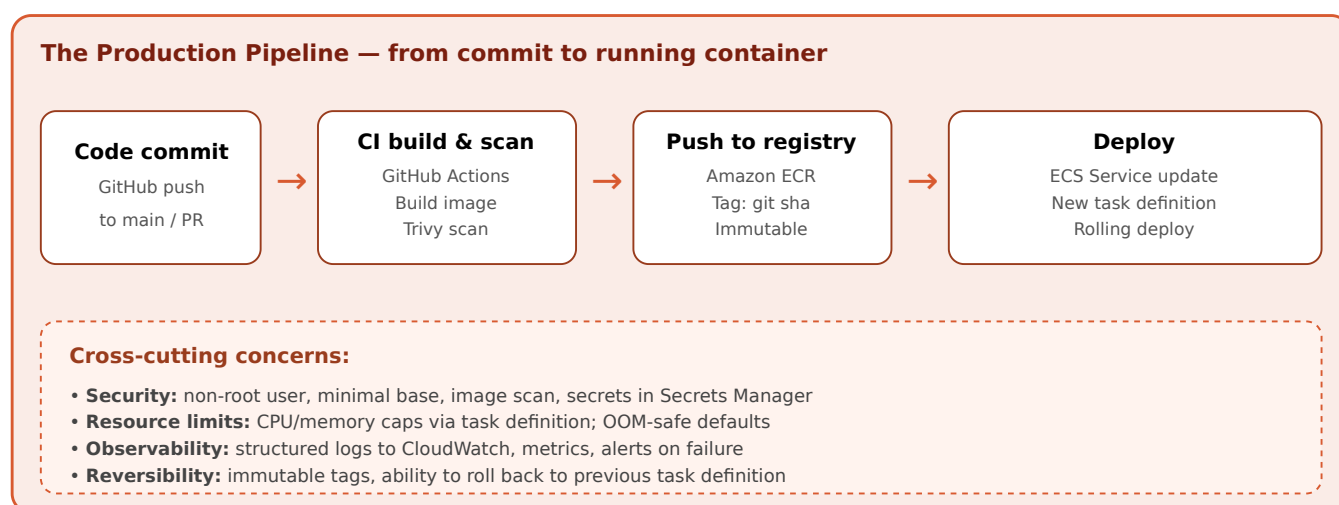


Figure 5.1 — The production pipeline. Every arrow is a place where things can break.

WHAT "PRODUCTION-READY" MEANS IN INTERVIEWS

- ▶ Container runs as a non-root user.
- ▶ Image is scanned for CVEs in CI; build fails on critical issues.
- ▶ Image is small and based on a minimal base (Alpine, distroless, or scratch).
- ▶ CPU and memory limits are set; not unlimited.
- ▶ Healthchecks are defined, both at container level and at orchestrator level.

- ▶ Logs go to a centralized destination (CloudWatch, ELK, Loki).
- ▶ Secrets come from a vault at runtime, not from environment in image.
- ▶ Deploys are reproducible — same git sha → same image → same deploy.

5.1 · Production-ready Dockerfile — the checklist

Most Dockerfiles online are dev-quality. Here is a production-grade Node.js example with every line earning its place.

```
# ---- Stage 1: build ----
FROM node:20-alpine AS builder
WORKDIR /build
COPY package*.json ./
RUN npm ci --omit=dev          # reproducible install, no dev deps
COPY . .
RUN npm run build              # compile / bundle

# ---- Stage 2: runtime ----
FROM node:20-alpine
ENV NODE_ENV=production
WORKDIR /app

# Create a non-root user (Alpine syntax)
RUN addgroup -S app && adduser -S app -G app
USER app

# Copy ONLY what's needed – leaves source/test/devdeps behind
COPY --from=builder --chown=app:app /build/node_modules ./node_modules
COPY --from=builder --chown=app:app /build/dist ./dist
COPY --from=builder --chown=app:app /build/package.json ./

EXPOSE 3000

# Healthcheck – orchestrators read this
HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \
  CMD wget -qO- http://localhost:3000/health || exit 1

# Use exec form so signals propagate (SIGTERM reaches the node process)
CMD ["node", "dist/server.js"]
```

The production checklist — go through this for every Dockerfile you ship

Check	Why it matters
Multi-stage build	Final image excludes build tools, dev dependencies, source files. Smaller, fewer CVEs.
Minimal base image	<code>alpine</code> (~5 MB), <code>distroless</code> (no shell), or <code>scratch</code> (empty). Smaller surface = fewer vulnerabilities.
Non-root <code>USER</code>	If the container is compromised, attacker doesn't get root inside. Defense in depth.
Pinned versions	<code>node:20.10.0-alpine3.19</code> , not <code>node:latest</code> . Reproducible builds.
<code>npm ci</code> not <code>npm install</code>	Strictly reproducible from lockfile. Required for hash-checking CI.
<code>.dockerignore</code>	Keep <code>node_modules</code> , <code>.git</code> , <code>.env</code> , <code>logs</code> out of build context. Smaller, faster builds, no secret leaks.
Layer ordering — copy lockfile before source	Caches dependency install layer between builds.
Use <code>COPY --chown</code> instead of <code>chmod</code> in RUN	Avoids creating extra layers and ensures correct ownership for non-root user.
<code>HEALTHCHECK</code> declared	Orchestrators (Compose, Swarm, ECS) can use it.
Exec form CMD: <code>CMD ["node", "x.js"]</code>	Shell form (<code>CMD node x.js</code>) wraps in <code>sh -c</code> , breaking signal forwarding. Container won't exit cleanly on SIGTERM.
No secrets in <code>ENV</code> or <code>ARG</code>	Image history exposes them forever.
No <code>ADD</code> when <code>COPY</code> works	<code>ADD</code> has surprising behaviors (URL fetching, auto-extracting tars). Use <code>COPY</code> .

The `.dockerignore` file you should always have

```
node_modules
npm-debug.log
.git
.gitignore
.env
.env.*
!.env.example
README.md
Dockerfile*
docker-compose*.yaml
.dockerignore
dist
build
coverage
*.log
.vscode
.idea
.DS_Store
```



Interview signal: *"Walk me through how you'd harden this Dockerfile for production."* — interviewers show a naive Dockerfile and want you to spot: missing multi-stage build, no `.dockerignore`, root user, `npm install` instead of `ci`, latest tag, missing healthcheck. Talk through them all without being prompted.

5.2 · Security best practices

Container security is layered. No single thing makes you "secure." It's a stack of defenses.

1. Don't run as root

```
# In Dockerfile
RUN addgroup -S app && adduser -S app -G app
USER app

# At runtime (overrides Dockerfile USER)
docker run --user 1000:1000 my-app
```

If your image still runs as root, run a quick check:

```
docker run --rm my-app id
# uid=0(root) gid=0(root) ← BAD
# uid=100(app) gid=100(app) ← GOOD
```

2. Use minimal base images

Base	Size	Trade-off
ubuntu:22.04	~80 MB	Familiar, lots of packages, more CVEs.
debian:bookworm-slim	~75 MB	Slimmer Debian.
alpine:3.19	~7 MB	Small. Uses musl libc (occasional compatibility issues with glibc-only binaries).
gcr.io/distroless/nodejs20	~70 MB	Has Node.js, nothing else — no shell, no package manager. Hardest to exploit.
scratch	0 bytes	Truly empty. Useful for fully-static binaries (Go). Nothing to exploit. Hardest to debug.

3. Scan images for vulnerabilities

```
# Trivy – the standard open-source scanner
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image my-app:latest

# Filter to high/critical only
trivy image --severity HIGH,CRITICAL my-app:latest

# Fail CI if any critical CVEs found
trivy image --severity CRITICAL --exit-code 1 my-app:latest

# Docker Scout (built into Docker Desktop)
docker scout cves my-app:latest
```


4. Read-only root filesystem

```
docker run --read-only --tmpfs /tmp --tmpfs /var/run my-app
# Container can't write anywhere except /tmp and /var/run (tmpfs).
# Stops most "drop a backdoor" exploits.
```

5. Drop unnecessary Linux capabilities

```
# Default: drop all caps, add back only what you need
docker run \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  nginx

# Without this, containers have many caps a normal user wouldn't
# Tools to inspect: capsh --print (inside container)
```

6. `--security-opt` **seccomp & AppArmor**

```
docker run --security-opt seccomp=default.json my-app
docker run --security-opt no-new-privileges my-app
# no-new-privileges blocks setuid escalation inside container
```

7. Never use `--privileged` unless you literally have to



--privileged gives the container full root access to the host kernel and devices. It's how container escapes happen. The only legitimate reasons are: running Docker-in-Docker (DinD) for CI, accessing GPU devices, or running low-level system tools. For everything else: never.

8. Secrets — final word

- **Never:** ENV in Dockerfile, COPY of .env, secrets in image tags, secrets logged.
- **OK for dev:** .env file in .gitignore.
- **Production:** AWS Secrets Manager / Parameter Store, Vault, sealed-secrets in K8s. Inject as env at boot OR mount as files via secrets mechanism.



Interview signal — guaranteed asked: "How do you handle secrets in a containerized app in production?" Strong answer: secrets are stored in AWS Secrets Manager (or Vault). At container boot, the entrypoint fetches them (using the IAM role of the ECS task / instance metadata) and exposes them to the app as env vars in the running process. They never appear in the image, in env files committed to git, or in logs. The container has IAM permissions for only its own secrets — least privilege.

5.3 · Resource management — CPU, memory, OOM

By default, a container can consume *all* the host's CPU and memory. In production, one bug in one container can take down everything else on the host. **Always set limits.**

Memory limits

```
docker run -m 512m my-app # hard cap
docker run --memory=512m --memory-swap=512m my-app # no swap allowed
docker run --memory-reservation=256m -m 512m my-app # soft floor + hard cap
```

When the container exceeds the limit, the kernel's **OOM killer** kills it. Symptom: container restarts; `docker inspect <id> --format='{{.State.OOMKilled}}'` returns true.

CPU limits

```
docker run --cpus="1.5" my-app # 1.5 cores worth of CPU
docker run --cpu-shares=512 my-app # relative weight (default 1024)
docker run --cpuset-cpus="0,1" my-app # pin to specific CPU cores
```

Monitor live resource usage

```
docker stats # live, top-like view of all containers
docker stats my-app # one container
docker stats --no-stream # one-shot snapshot
```

In Compose

```
services:
  api:
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512m
        reservations:
          cpus: "0.5"
          memory: 256m
```



Going deeper — cgroups: Docker uses Linux cgroups (control groups) to enforce these limits. cgroup v2 (default on modern distros) is more capable than v1. When you set `-m 512m`, Docker writes to `/sys/fs/cgroup/.../memory.max`. Knowing this answers "how does Docker actually enforce memory limits?" in interviews.



Interview signal: "What happens when a container exceeds its memory limit?" Answer: the Linux kernel OOM-killer terminates a process inside the container (usually PID 1, which kills the container). The container's `State.OOMKilled` flag is true. If you have a restart policy, it restarts. Memory limits are hard — exceeding by 1 byte over the limit kills it.

5.4 · Logging, monitoring & debugging

Where container logs live

Anything your app writes to `stdout` or `stderr` is captured by Docker's logging driver. By default, it's stored as JSON files on the host (`/var/lib/docker/containers/<id>/<id>-json.log`).

```
docker logs my-app                # all logs since start
docker logs -f my-app             # follow (tail -f)
docker logs --tail 100 my-app
docker logs --since 10m my-app
docker logs --timestamps my-app
```

Log drivers — switch the destination

```
# Default — JSON files on host
docker run --log-driver json-file my-app

# Rotate logs to avoid filling disk (CRITICAL in production)
docker run \
  --log-driver json-file \
  --log-opt max-size=10m \
  --log-opt max-file=3 \
  my-app

# Ship logs to CloudWatch directly
docker run --log-driver awslogs \
  --log-opt awslogs-region=ap-south-1 \
  --log-opt awslogs-group=my-app-prod \
  my-app

# Other drivers: syslog, journald, fluentd, gelf, splunk, none
```



Pitfall — unbounded logs filling disk: The default JSON-file driver has *no size limit*. A chatty app can write GB per day. Always set `max-size` + `max-file` (or set defaults in `/etc/docker/daemon.json`).

Structured logging — what apps should emit

Plain text logs are a debugging nightmare at scale. Apps should emit JSON:

```
{
  "ts": "2026-05-13T10:00:00Z",
  "level": "info",
  "msg": "request",
  "method": "GET",
  "path": "/api/users",
  "duration_ms": 42
}
```

This is parseable by CloudWatch Insights, Loki, ELK, Datadog — everything. Use libraries like `pino` (Node), `structlog` (Python), `zap` (Go).

Real-time stats & events

```
docker stats          # CPU/mem/net/io per container
docker events         # stream of all Docker events on the host
docker top my-app     # processes inside the container
docker inspect my-app # full state, env, mounts, network, exit code
```

Debugging a crashing container — the standard playbook

1. `docker logs <id>` — the obvious first step.
2. `docker inspect <id> --format='{{.State.ExitCode}} {{.State.Error}}'` — what did the kernel/runtime say?
3. `docker inspect <id> --format='{{.State.OOMKilled}}'` — out of memory?
4. Override the entrypoint to drop into a shell: `docker run -it --entrypoint sh my-app` — then run things manually.
5. `docker stats` while reproducing — watching for memory/CPU spikes.
6. If the container's filesystem matters: `docker cp <id>:/path /tmp/inspect` to extract files.

5.5 · Docker in CI/CD with GitHub Actions

You already have GitHub Actions experience from your earlier work. Now add Docker. Standard pattern: on push to main, build the image, scan it, push it to a registry, deploy.

Minimal build-and-push workflow

```
# .github/workflows/docker.yml
name: Docker Build & Push

on:
  push:
    branches: [main]

jobs:
  build-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write    # for GitHub Container Registry

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}

      - name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${GITHUB_REPOSITORY}:latest
            ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_SHA}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

What's happening here:

- **docker/setup-buildx-action**: enables BuildKit — faster builds, better caching, multi-arch.
- **docker/login-action**: logs in to the registry. GITHUB_TOKEN is built-in.
- **docker/build-push-action**: one-step build + push.
- **cache-from/cache-to gha**: caches layers in GitHub Actions cache — speeds rebuilds drastically.
- **Two tags**: latest for "current main" and the git sha for immutable, reproducible deploys.

Add image scanning before push

```
- name: Build image (local first)
  uses: docker/build-push-action@v6
  with:
    context: .
    push: false
    load: true
    tags: my-app:scan

- name: Trivy scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: my-app:scan
    format: table
    exit-code: 1
    severity: HIGH,CRITICAL
    ignore-unfixed: true

- name: Push (only if scan passed)
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ...
```

Multi-arch builds (ARM + x86)

Apple Silicon, AWS Graviton (cheaper ARM EC2), and Raspberry Pi all need ARM images. BuildKit + buildx makes this trivial:

```
- name: Set up QEMU                                # cross-arch emulation
  uses: docker/setup-qemu-action@v3

- name: Set up Buildx
  uses: docker/setup-buildx-action@v3

- name: Build & push multi-arch
  uses: docker/build-push-action@v6
  with:
    context: .
    platforms: linux/amd64,linux/arm64    # both!
    push: true
    tags: ghcr.io/me/my-app:latest
```



Interview signal: "How would you ensure that a Dockerfile in a PR doesn't introduce vulnerabilities?" Answer: a CI workflow that builds the image and runs Trivy (or Snyk/Dockle/Grype) with `--severity HIGH,CRITICAL --exit-code 1`. The PR check fails until vulnerabilities are addressed (fixed, base image upgraded, or explicitly waived in a tracked ignore file). Mention also that the base image should be regularly rebuilt to pull patches even when source hasn't changed.

5.6 · Amazon ECR — private registry on AWS

For deploying to AWS infrastructure, ECR (Elastic Container Registry) is the natural choice. It's a managed Docker registry integrated with IAM.

Concepts

- **Repository** — one per image (e.g., `my-app/api`). Holds many tags.
- **Private vs Public** — ECR has both. You'll almost always use private.
- **IAM-based auth** — no static creds. Get a short-lived token from `aws ecr get-login-password`.
- **Lifecycle policies** — auto-delete old images to save cost (e.g., "keep last 10 of main, delete untagged after 7 days").
- **Image scanning** — ECR has built-in basic scanning (free) and "enhanced scanning" (paid, by Amazon Inspector).

Push to ECR — manually

```
# Create the repo (one time)
aws ecr create-repository --repository-name my-app/api --region ap-south-1

# Get an auth token and login (12-hour validity)
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin \
    <account-id>.dkr.ecr.ap-south-1.amazonaws.com

# Tag your local image with the ECR URI
docker tag my-app:latest \
  <account-id>.dkr.ecr.ap-south-1.amazonaws.com/my-app/api:v1.0.0

# Push
docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/my-app/api:v1.0.0
```

Push to ECR — from GitHub Actions (the right way)

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/github-actions-ecr
    aws-region: ap-south-1

- name: Login to ECR
  id: ecr-login
  uses: aws-actions/amazon-ecr-login@v2

- name: Build & push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ${ steps.ecr-login.outputs.registry }/my-app/api:${ github.sha }
      ${ steps.ecr-login.outputs.registry }/my-app/api:latest
```



OIDC, not access keys. The role-to-assume setup uses GitHub's OIDC provider to assume an IAM role temporarily — no long-lived AWS access keys stored in GitHub secrets. This is the modern AWS-recommended approach. Worth understanding for any AWS-DevOps interview.

Lifecycle policy — automatic cleanup

```
{
  "rules": [{
    "rulePriority": 1,
    "description": "Keep last 10 images tagged with sha",
    "selection": {
      "tagStatus": "tagged",
      "tagPrefixList": [""],
      "countType": "imageCountMoreThan",
      "countNumber": 10
    },
    "action": { "type": "expire" }
  }, {
    "rulePriority": 2,
    "description": "Delete untagged after 7 days",
    "selection": {
      "tagStatus": "untagged",
      "countType": "sinceImagePushed",
      "countUnit": "days",
      "countNumber": 7
    },
    "action": { "type": "expire" }
  }]
}
```


5.7 · AWS ECS — running containers in the cloud

ECS (Elastic Container Service) is AWS's container orchestrator. Two flavours: **ECS on EC2** (you manage instances) and **ECS on Fargate** (serverless — AWS runs the infra). Fargate is what 99% of new projects use.

The four ECS concepts

Concept	Role
Cluster	A logical grouping of tasks/services. With Fargate, mostly just an organizational unit.
Task Definition	The "recipe" — which image, how much CPU/memory, env vars, ports, log driver, IAM role. Versioned (revision number). Analogous to a Compose service definition.
Task	A running instance of a task definition. Like a container (or container group — a task can have multiple containers, like a K8s pod).
Service	Keeps N tasks running (auto-restarts crashed ones), handles rolling updates, optionally registers with a load balancer.

A minimal task definition (JSON)

```
{
  "family": "my-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",           # 0.5 vCPU
  "memory": "1024",       # 1 GB
  "executionRoleArn": "arn:aws:iam::ACCT:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::ACCT:role/my-app-task-role",
  "containerDefinitions": [{
    "name": "api",
    "image": "123456789012.dkr.ecr.ap-south-1.amazonaws.com/my-app/api:v1.0.0",
    "essential": true,
    "portMappings": [{ "containerPort": 3000, "protocol": "tcp" }],
    "environment": [
      { "name": "NODE_ENV", "value": "production" }
    ],
    "secrets": [
      { "name": "DB_PASSWORD",
        "valueFrom": "arn:aws:secretsmanager:ap-south-1:ACCT:secret:db-pass-XYZ" }
    ],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/my-app",
        "awslogs-region": "ap-south-1",
        "awslogs-stream-prefix": "api"
      }
    },
    "healthCheck": {
      "command": ["CMD-SHELL", "curl -f http://localhost:3000/health || exit 1"],
      "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 30
    }
  }]
}
```

Translate this to your mental model:

- `family` = service name (like the key under `services:` in Compose).
- `cpu` / `memory` = task-level limits. Fargate has a fixed set of valid combinations (256/512MB up to 16 vCPU/120GB).
- `executionRoleArn` = the role ECS uses to pull the image from ECR & send logs to CloudWatch.
- `taskRoleArn` = the role your *application code* assumes (e.g., to read from S3). This is how containers get AWS credentials without env vars.
- `secrets` = inject from Secrets Manager / Parameter Store as env vars at boot. **No secrets in the image.**

Two ways to deploy

CLI / SDK (good for learning, manual ops):

```
aws ecs register-task-definition --cli-input-json file://taskdef.json
aws ecs update-service --cluster prod --service api \
  --task-definition my-app:5 --desired-count 3
```

From GitHub Actions (the production way):

```
- name: Render task definition
  id: render-td
  uses: aws-actions/amazon-ecs-render-task-definition@v1
  with:
    task-definition: taskdef.json
    container-name: api
    image: ${ steps.ecr-login.outputs.registry }}/my-app/api:${ github.sha }

- name: Deploy to ECS
  uses: aws-actions/amazon-ecs-deploy-task-definition@v2
  with:
    task-definition: ${ steps.render-td.outputs.task-definition }
    service: api
    cluster: prod
    wait-for-service-stability: true
```

The full pipeline — what your end-state CI/CD looks like

1. Push to main → GitHub Actions triggers.
2. Build Docker image with multi-stage Dockerfile.
3. Trivy scan → fail on critical CVEs.
4. Push image to ECR with tags: :latest and :<git-sha>.
5. Render a new task definition referencing the new image.
6. Update the ECS service with the new task definition.
7. ECS does a rolling deploy: starts new tasks, drains old, swaps load balancer targets.
8. If any task fails healthcheck, the service auto-rolls back.



Interview signal — this is the gold-standard answer for any "describe a deployment pipeline" question. Walk through these 8 steps with confidence and you've immediately demonstrated production-level DevOps thinking.

5.8 · Hands-on labs — Level 5

Lab 5.1 — Harden a Dockerfile

Goal: take a naive Dockerfile, harden it step-by-step, see image size shrink.

Before (~900 MB image):

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD npm start
```

After (~150 MB image, non-root, healthcheck, multi-stage):

```
FROM node:20-alpine AS builder
WORKDIR /build
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build

FROM node:20-alpine
ENV NODE_ENV=production
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY --from=builder --chown=app:app /build/node_modules ./node_modules
COPY --from=builder --chown=app:app /build/dist ./dist
COPY --from=builder --chown=app:app /build/package.json ./
USER app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

Measure: `docker images` before/after. Confirm non-root: `docker run --rm my-app id` shows `uid=100(app)`.

Lab 5.2 — Image vulnerability scan with Trivy

Goal: scan a deliberately old image, see CVEs, upgrade base, re-scan, see improvement.

```
# Scan an intentionally old image
docker pull node:14
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image node:14

# Compare with a current one
docker pull node:20-alpine
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image node:20-alpine
```

Observation: the number of critical CVEs drops by 90%+ moving to a current, minimal base. This is why "use a recent, slim base" is a security recommendation, not just a size one.

Lab 5.3 — GitHub Actions: build and push to GitHub Container Registry

Goal: set up a working CI pipeline end-to-end. Verify image appears in GHCR after a push.

Steps:

1. In a GitHub repo with your hardened Dockerfile, create `.github/workflows/docker.yml` with the workflow from section 5.5.
2. Push to main. Watch Actions tab.
3. After success, check your packages tab — image should be visible at `ghcr.io/<you>/<repo>:latest`.
4. Pull it back from another machine to confirm: `docker pull ghcr.io/you/repo:latest`.

Add Trivy scan step. Push a deliberate vulnerability (use an old base image). Watch the CI fail. Fix it. Watch it pass.

Lab 5.4 — Push to Amazon ECR

Goal: end-to-end push to ECR using OIDC from GitHub Actions.

Pre-reqs: AWS Free Tier account, AWS CLI configured locally.

1. Create ECR repo via console or CLI: `aws ecr create-repository --repository-name lab/my-app`.
2. Manual push first to confirm setup: `aws ecr get-login-password | docker login --username AWS --password-stdin ...`; then tag & push.
3. Set up GitHub→AWS OIDC trust: in AWS IAM, create an identity provider for `token.actions.githubusercontent.com`; create a role that GitHub Actions can assume with permissions to push to your ECR repo only.
4. Update your workflow to use `aws-actions/configure-aws-credentials` with `role-to-assume`.
5. Push, verify image in ECR console.

This is high-leverage practice — the OIDC pattern is what real AWS teams use.

Lab 5.5 — Deploy to ECS Fargate

Goal: get an actual container running in AWS, reachable on the internet.

1. Create an ECS cluster (Fargate-only) via console.
2. Create a task definition from your image in ECR (use the JSON from section 5.7 as template).
3. Create an ECS service with desired count = 1, attached to a default-VPC public subnet, with security group allowing inbound port 3000.
4. Wait for task to reach RUNNING state. Get its public IP from console.
5. `curl http://<public-ip>:3000` — get a real response from your container running in AWS.
6. Update task definition (e.g., change an env var), update service, watch a rolling deploy.
7. For production-grade, add an Application Load Balancer in front — service registers tasks as targets.



AWS cost note: Fargate is metered by vCPU-hour + GB-hour. A tiny task running 24/7 costs about \$5-10/month. Tear down after the lab to avoid surprises. Set up an AWS Budget alert at \$5.

Lab 5.6 — End-to-end pipeline (capstone)

Goal: combine everything. Push to GitHub → image scanned → pushed to ECR → ECS service updated → new version live.

This is the showpiece lab. Most product-company DevOps interviews will ask you to *describe* exactly this pipeline. After Lab 5.6, you can describe it from experience.

Acceptance criteria:

- A code change on main triggers GitHub Actions automatically.
- Build runs, Trivy scans, fails on critical CVEs.
- Image pushes to ECR with two tags: latest + <git-sha>.
- Task definition is rendered with the new image tag.
- ECS service updates; new tasks start; old tasks drain.
- The change is visible at your public endpoint within ~5 minutes of `git push`.
- Rollback works: re-deploy the previous git sha's task definition.

Get this working. Put it on your CV. Demo it in interviews. **This single lab is worth more than every other Docker tutorial combined.**

5.9 · Common pitfalls — production & CI/CD



Running as root in production. If something compromises your container, the attacker has root inside, and any container escape gets them host root. Always non-root.



Using :latest tags in production deploys. Two deploys of "latest" can yield different images. You can't roll back to a specific known version. Always pin to git sha or semver.



No image scan in CI. First time you find out about CVEs is when a customer's security team scans your stuff. Trivy or Snyk in CI, fail-fast.



Secrets in env vars baked into images. Image history exposes them. Even private registry images leak. Use Secrets Manager / Vault.



No memory/CPU limits. One leaky container takes down the entire host. ECS/K8s require limits anyway; in plain Docker, set them.



No log rotation. Default json-file driver has no limit. Disk fills, container can't write, host hangs. Always max-size + max-file.



Building images on developer laptops and pushing. No audit trail, "works on my laptop" surface area. Always build in CI; developers can't push to prod registries.



Forgetting to delete old ECR images. Costs add up; storage isn't free. Lifecycle policies clean automatically.



Hard-coding region/account IDs in workflows. Use GitHub variables/secrets and environment-scoped values. Workflow should be promotable from dev → staging → prod with just env changes.



Long-lived AWS access keys in GitHub secrets. Get rotated, get leaked. Use OIDC + assume-role. No static creds in repos in 2026.

5.10 · Interview Q&A — Level 5 (production & CI/CD)

BASIC · 15 QUESTIONS

Q. What is a multi-stage build and why does it matter?

A. A Dockerfile with multiple FROM statements. Earlier stages can build artifacts (compile code, install dev deps); final stage COPY --from only the artifacts. The result is a smaller, more secure runtime image without build tooling.

Q. Why should containers run as non-root?

A. Defense in depth. If the app is compromised (RCE, etc.), the attacker is constrained to a non-privileged user inside the container. Any container escape still requires further privilege escalation. Many compliance frameworks require it.

Q. Difference between npm install and npm ci?

A. npm ci reads package-lock.json only — fully reproducible, faster, fails if lockfile is out of sync. npm install may update the lockfile. Use ci in Dockerfiles and CI.

Q. What does the HEALTHCHECK instruction in a Dockerfile do?

A. Defines a command that Docker runs periodically to test if the container is healthy. Orchestrators (Compose, Swarm, ECS) can read this status to decide whether to route traffic, restart, or fail deploys.

Q. What's the difference between CMD shell form and exec form?

A. Shell form (CMD node app.js) wraps the command in /bin/sh -c — signals like SIGTERM go to the shell, not your app, so graceful shutdown breaks. Exec form (CMD ["node", "app.js"]) runs the binary directly as PID 1. Use exec form.

Q. What does the .dockerignore file do?

A. Lists files/directories excluded from the build context sent to the Docker daemon. Smaller context → faster builds. Also prevents secrets, dev files, and bloat from leaking into images.

Q. What's the simplest way to scan an image for vulnerabilities?

A. Trivy: trivy image my-app:latest. Or Docker Scout: docker scout cves my-app:latest. Both are free and CI-friendly.

Q. How do you set a memory limit on a container?

A. docker run -m 512m my-app. Or in Compose under deploy.resources.limits.memory: 512m. The Linux kernel enforces it via cgroups; OOM-killer terminates if exceeded.

Q. What does it mean when `State.OOMKilled` is true after a container exit?

A. The Linux kernel's OOM killer terminated the container because its memory usage exceeded the configured limit (or the host ran out of memory). Either raise the limit or fix the memory leak.

Q. How do you see real-time CPU/memory usage of all running containers?

A. `docker stats`. Top-like live view. `--no-stream` for a single snapshot.

Q. What is Amazon ECR?

A. AWS's managed private Docker registry. IAM-based auth, integrated with ECS/EKS/Lambda, optional image scanning, lifecycle policies for automatic cleanup.

Q. What is an ECS task definition?

A. A versioned JSON document describing how to run one or more containers: image, CPU, memory, env vars, ports, networking, IAM role, log config. The "recipe" — a task is a running instance of it.

Q. Difference between ECS on EC2 and ECS on Fargate?

A. EC2 mode — you provision and manage the underlying EC2 instances; you pay for the instances. Fargate — AWS manages the infra; you pay per vCPU-hour and GB-hour of task usage. Fargate has higher per-unit cost but no instance management overhead; usually the right default.

Q. What's the role of `docker/build-push-action` in GitHub Actions?

A. A single action that builds an image (using BuildKit/buildx), tags it, and pushes it to a registry. Supports layer caching (`cache-from/cache-to`), multi-arch builds, and integrates with `docker/login-action` for registry auth.

Q. What does `docker logs --tail 100 -f my-app` do?

A. Shows the last 100 lines of the container's stdout/stderr and follows new output as it appears (`-f`).

ADVANCED · 10 QUESTIONS

Q. What's the difference between an `executionRoleArn` and a `taskRoleArn` in ECS?

A. `executionRoleArn` is the role *ECS itself* uses to pull images from ECR and send logs to CloudWatch — infrastructure plumbing. `taskRoleArn` is the role your *application code* assumes inside the container to access AWS services like S3 or DynamoDB. Separating them follows least privilege.

Q. How do you implement zero-downtime rolling deploys in ECS?

A. The ECS service is configured with deployment settings: `minimumHealthyPercent: 100` and `maximumPercent: 200` (during deploy, both old and new tasks run; old aren't drained until new are healthy). Combined with healthchecks + load balancer target group health checks, the service replaces tasks one at a time.

Q. What's the OIDC pattern for GitHub Actions → AWS, and why use it instead of static keys?

A. GitHub Actions provides each workflow run with a short-lived OIDC JWT. AWS IAM trusts GitHub as an OIDC identity provider and allows a specific repo/branch to assume an IAM role. The role grants only the minimum permissions needed. Benefit: no long-lived AWS access keys stored as GitHub secrets — nothing to leak or rotate.

Q. How does BuildKit's caching differ from classic Docker build caching?

A. Classic caches per layer based on previous layer + instruction. BuildKit adds: parallel stage builds (independent stages run concurrently), inline cache export/import (cache pushed to a registry or stored externally for sharing across CI runs), mount-based caches (`--mount=type=cache` for things like `~/ .cache/pip`), and content-addressed caching. Much faster CI rebuilds.

Q. What does `--mount=type=cache` in a Dockerfile do?

A. A BuildKit feature. Creates a persistent cache mount during build that survives between builds (not in the final image). Used for package manager caches: `RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt`. Massively speeds up rebuilds.

Q. When would you use a distroless image vs Alpine?

A. Distroless (`gcr.io/distroless/...`) has no shell or package manager — minimum attack surface, hard to exploit, hard to debug. Use for high-security workloads or known-stable services. Alpine has a shell and `apk` — easier to debug, can install runtime tools if needed, slightly larger surface. Default to Alpine; escalate to distroless for hardened services.

Q. Why does the order of instructions in a Dockerfile matter for caching?

A. Docker invalidates the cache from the first changed layer onward. So put rarely-changing things (base image, system packages) first; frequently-changing things (your code) last. Specifically: `COPY package*.json ./ && RUN npm ci` before `COPY . .`, so deps reinstall only when lockfile changes.

Q. How would you build a multi-arch image?

A. Use BuildKit/buildx with QEMU for emulation. In GitHub Actions: `docker/setup-qemu-action` + `docker/setup-buildx-action`, then `build-push-action` with platforms: `linux/amd64,linux/arm64`. Result is a manifest list — the registry serves the right architecture to each puller.

Q. What's the difference between AWS Secrets Manager and Parameter Store for container secrets?

A. Secrets Manager: built for secrets, supports rotation, costs per secret + per API call. Parameter Store: general parameter storage, free tier exists, simpler. For DB passwords / API keys that need rotation, use Secrets Manager. For non-sensitive config or basic secrets, Parameter Store (SecureString) is cheaper. ECS task definitions integrate with both via the `secrets` array.

Q. How do you debug a container that crashes immediately on start in production?

A. (1) `docker logs` / CloudWatch logs — what did it say before dying? (2) `docker inspect` / ECS task details — exit code, OOMKilled flag, stopped reason. (3) Pull the same image locally, run with `-it --entrypoint sh`, manually invoke the entrypoint — find where it fails. (4) Compare config: env vars, mounted secrets, IAM permissions — production-only env vars often differ from local. (5) For ECS: check the task's execution role can pull the image and write logs; missing permissions are the #1 reason for "stuck in PROVISIONING" failures.

SCENARIO · 10 QUESTIONS

Q. A deployment to ECS keeps failing — new tasks start, fail health checks, then get killed. Where do you look?

A. (1) CloudWatch logs for the new task — what is the app saying? (2) Task definition: is the healthcheck command correct? Path right, port right? (3) Security group: can the load balancer's health check reach the task's port? (4) Application startup time: does `startPeriod` give enough grace? (5) Memory: was OOMKilled in stopped reason? (6) Image: did the new image actually pull, or is it failing because the tag doesn't exist?

Q. Your CI pipeline pushes `:latest` on every main commit. A bad commit causes prod issues. How do you roll back?

A. You can't cleanly — `:latest` already points to the broken image. **Lesson: never deploy `:latest` to prod.** Fix going forward: tag images with git SHA, deploy by SHA, rollback = re-deploy the previous SHA's task definition. Recovery now: identify last-known-good image (from ECR history or task definition history), re-tag it as `:latest`, force a new ECS deployment.

Q. An image vulnerability scan in CI fails with 50 CVEs. Devs say "they're all in the base image, not our code, just ignore them." Your call?

A. Don't ignore wholesale. Audit: how many are actually critical and exploitable in this app's runtime? Most can be fixed by upgrading the base image (`node:20.10` → `20.12`). For unfixed CVEs in the base, document a waiver with expiry date — re-scan monthly. The trap is silently disabling the scan; that's how teams ship known-vulnerable images.

Q. Image size is 1.2 GB. How do you systematically reduce it?

A. (1) `docker history my-app` — what layer added the most? (2) Use a multi-stage build to leave build tools behind. (3) Switch base from `node:20` (~900MB) to `node:20-alpine` (~150MB). (4) Add `.dockerignore` — don't ship `node_modules`, `.git`, tests. (5) Combine RUN steps to avoid extra layers; clean apt cache. (6) For Go/Rust: `distroless` or `scratch` — single binary, ~10MB images.

Q. Your team is debating: ECS Fargate or EKS (Kubernetes on AWS)? You're 8 engineers, one service.

A. Fargate. Honestly. Kubernetes is overkill at your scale — the operational burden of running K8s well dwarfs the benefit until you're orchestrating dozens of services with complex routing. Fargate gives you task-level isolation, autoscaling, rolling deploys, IAM integration — sufficient for 95% of teams. Move to EKS when you have specific K8s-only requirements (custom operators, service mesh, multi-cloud, complex stateful workloads).

Q. A container in production has memory usage steadily climbing for 48 hours, then OOMKilled, restart, climb again. Diagnose.

A. Memory leak. Container restarts mask the symptom but don't fix it. (1) Profile the app: memory profiling tools for the language (heapdump for Node, tracemalloc for Python). (2) Check for common causes: caches with no eviction, unclosed DB connections, listener leaks. (3) Short-term: increase memory limit to give more headroom + add alerting on restarts. (4) Long-term: fix the leak. Setting a higher memory limit isn't a fix.

Q. A developer needs to access production secrets locally for debugging. How do you give access without leaking credentials?

A. Don't share the secrets directly. Give the dev temporary IAM permissions to read from Secrets Manager themselves (via SSO + role assumption, with audit logging). They fetch fresh at debug time; nothing's stored on their laptop. For very sensitive prod data: do the debug session over a screen-shared bastion host, no local copy. For repeated access: build a non-prod replica with synthetic data so prod access isn't needed.

Q. Your ECS service is "stable" but customer reports show it's slow under load. How do you investigate?

A. (1) Check container-level: CloudWatch Container Insights for CPU/memory pressure. CPU pinned at limit = throttling. (2) Check service-level: task count = desired count? Auto-scaling configured? (3) Check downstream: DB IOPS, connection pool saturation. (4) Application-level: APM (X-Ray, Datadog) shows where latency lives. (5) Fix: scale out (more tasks) or scale up (more CPU/mem per task), or fix the bottleneck — whatever the data points to.

Q. A teammate proposes building images on a self-hosted GitHub Actions runner to "speed things up." What concerns do you raise?

A. Self-hosted runners on shared infra have real security risks: untrusted PR code runs on a machine that may have prod credentials cached, network access to internal services, persistent state from previous runs. If you go this route: ephemeral runners (fresh VM per job), strict network isolation, no PR builds from forks. For speed, often simpler wins: better Docker layer caching (cache-from gha), parallel jobs, larger GitHub-hosted runners (paid tier).

Q. Walk me through your end-to-end Docker pipeline from commit to production. (This is the question.)

A. A developer pushes a feature branch and opens a PR. GitHub Actions runs: build the Docker image with the hardened multi-stage Dockerfile, run unit and integration tests inside it, scan with Trivy failing on critical CVEs, lint the Dockerfile with hadolint. On PR merge to main, a second workflow runs: rebuild, retag with the git SHA, push to ECR with both `:latest` and `:<sha>`, render a new ECS task definition pointing at the new image SHA, call ECS update-service, wait for service stability. ECS does a rolling deploy gated by health checks on the new tasks; if any fail, the service auto-rolls back to the previous revision. Secrets are pulled at task boot from AWS Secrets Manager via the task IAM role — never in the image. The whole pipeline takes ~6 minutes; rollback to any prior SHA is one re-deploy of the older task definition. *This is the answer that ends interviews.*

APX Appendix — Reference & Plan

The references you'll come back to. The plan that makes the 4-6 weeks real.

A.1 · Master command cheatsheet

Everything from Levels 3, 4, 5 — one page. Print it. Stick it on your wall.

Networks

docker network ls

List networks

docker network create app-net

Create user-defined bridge

docker network inspect app-net

Show containers, subnet, IPs

docker network connect app-net api

Attach container to network

docker network prune

Delete unused networks

docker run --network app-net ...

Start on a network

Ports

docker run -p 8080:80 nginx

host:container

docker run -p 127.0.0.1:8080:80 nginx

Bind to localhost only

docker run -P nginx

Publish all EXPOSEd ports randomly

docker port my-web

Show current mappings

Volumes

docker volume create pgdata

Create named volume

docker volume inspect pgdata

Show mountpoint

docker volume prune

Delete unused

docker run -v pgdata:/data ...

Mount named volume

docker run -v \$(pwd):/app ...

Bind mount (dev)

docker run --tmpfs /tmp ...

RAM mount

Compose

docker compose up -d

Start stack (detached)

docker compose down

Stop & remove (keeps volumes)

docker compose down -v

⚠ Also deletes volumes

docker compose ps / logs -f

List / follow logs

docker compose exec api sh

Shell into running service

docker compose run --rm api npm test

One-shot in fresh container

docker compose config

Print merged YAML

docker compose up -d --scale api=3

Scale stateless service

docker compose -f a.yml -f b.yml up

Merge multiple files

docker compose --profile debug up

Include profile-tagged services

Production / Resources

docker run -m 512m --cpus="1.0" ...

Memory + CPU limits

docker run --user 1000:1000 ...

Run as non-root

docker run --read-only --tmpfs /tmp

Read-only FS

docker run --cap-drop ALL ...

Drop Linux capabilities

docker stats

Live resource usage

docker logs --tail 100 -f x

Tail logs

**docker inspect x --
format='{{.State.OOMKilled}}'**

Was it OOMKilled?

trivy image my-app:latest

Scan for CVEs

ECR / ECS

**aws ecr get-login-password | docker login --
username AWS --password-stdin <uri>**

Auth to ECR

**aws ecr create-repository --repository-name
my-app/api**

Create ECR repo

**aws ecs register-task-definition --cli-
input-json file://taskdef.json**

Register new TD

**aws ecs update-service --cluster X --service
Y --task-definition my-app:5**

Roll out new TD

Debug toolkit

**docker run --rm -it --network app-net
nicolaka/netshoot**

Debug network from inside

docker exec -it api sh

Shell into running container

docker cp x:/path /tmp/here

Copy file from container

docker run -it --entrypoint sh my-app

Override broken entrypoint

docker history my-app

See image layer sizes

docker events

Stream of all Docker events

A.2 · Top-30 most-asked Docker interview questions

The questions you must be able to answer cold, in 30 seconds each, in any product-company round.

Q. What's the difference between a container and a VM?

A. Both isolate workloads. A VM virtualizes hardware — full guest OS, hypervisor, GBs of overhead, slow to start. A container shares the host kernel and isolates processes via namespaces + cgroups — MBs of overhead, starts in seconds. Containers are denser; VMs offer stronger isolation.

Q. What's the difference between an image and a container?

A. An image is a static, immutable filesystem snapshot with metadata about how to run it. A container is a running (or stopped) process instance launched from an image. Image is class; container is instance. You can have many containers from one image.

Q. Walk me through what happens when you run `docker run nginx`.

A. (1) Docker CLI sends a request to the daemon. (2) Daemon checks if `nginx:latest` is local; if not, pulls from Docker Hub. (3) Creates a new container layer on top of the image. (4) Sets up network namespace, attaches to default bridge. (5) Starts the container's main process (`nginx`). (6) Streams output to the CLI.

Q. What is a Dockerfile?

A. A text file with instructions to build an image. Each instruction (`FROM`, `COPY`, `RUN`, etc.) creates an immutable layer. Build with `docker build -t name:tag ..`

Q. What's a multi-stage build and when do you use it?

A. A Dockerfile with multiple `FROM` stages. Build artifacts in early stages; copy only what's needed into the final, minimal stage. Smaller, more secure runtime images. Always use for compiled languages and Node.js apps.

Q. How does Docker layer caching work, and how do you optimize Dockerfiles for it?

A. Each Dockerfile instruction produces a cached layer; cache invalidates from the first changed instruction onward. Put rarely-changing things (base, system deps) first; frequently-changing things (your source code) last. Copy lockfile + install deps before copying full source.

Q. What's the difference between `CMD` and `ENTRYPOINT`?

A. `ENTRYPOINT` defines the executable; `CMD` provides default arguments. If both: container runs `ENTRYPOINT + CMD`. `docker run image cmd` overrides `CMD` but not `ENTRYPOINT`. Use `ENTRYPOINT` for "this image is this binary"; `CMD` for tweakable defaults.

Q. Why should you never use the default bridge network?

A. No automatic DNS — containers can only reach each other by IP, and IPs change on restart. Weak isolation between containers. Always `docker network create` a user-defined bridge for any real app.

Q. How does container-to-container DNS work?

A. On a user-defined network, Docker runs an embedded DNS server at 127.0.0.11 inside each container. It resolves other container names on the same network to current IPs, automatically.

Q. Why doesn't localhost work for connecting to another container?

A. Each container has its own network namespace. `localhost` refers to the container itself. To reach another container, use its name (DNS on user-defined network) or the host's IP.

Q. Difference between EXPOSE in Dockerfile and -p at runtime?

A. EXPOSE is documentation/metadata — it does not publish anything. `-p` actually opens a host port and forwards to the container. EXPOSE is consumed by tools like `-P`.

Q. Difference between volume, bind mount, and tmpfs?

A. Volume — Docker-managed, portable, lives under `/var/lib/docker/volumes/`, used for production data. Bind mount — direct mapping of any host path; great for dev (live reload) and config injection. Tmpfs — in-memory, dies with container; for secrets and scratch data.

Q. How do you persist database data across container restarts?

A. Mount a named volume to the database's data directory. `docker run -v pgdata:/var/lib/postgresql/data postgres`. Even if you delete and recreate the container, the volume — and data — survives.

Q. How do you handle secrets in containers?

A. Never bake into images. Dev — env file in `.gitignore`. Prod — fetch from Secrets Manager / Vault at runtime via the container's IAM role; inject as env vars or mounted files. Audit log every access.

Q. What's Docker Compose and when do you use it?

A. YAML-driven tool for running multi-container applications declaratively. Use for local dev, CI test stacks, single-host deployments. Don't use for multi-host or auto-scaling production — that's K8s/ECS.

Q. Why is `depends_on` alone usually not enough, and what's the fix?

A. It waits for the dependency container to *start*, not for the app inside to be *ready*. Fix: define a healthcheck on the dependency, then use `depends_on: { condition: service_healthy }`.

Q. What's the difference between `docker compose exec` and `docker compose run`?

A. `exec` attaches to an already-running service container. `run` spawns a fresh container from the service config to run a one-off command. Use `exec` for live debugging; `run --rm` for tests/migrations/one-shots.

Q. How do you handle dev vs prod configs in Compose?

A. Layered files: base `compose.yml` with shared config, then `compose.dev.yml` or `compose.prod.yml` with overrides. Merge with `-f base.yml -f env.yml`. Verify with `docker compose config`.

Q. Why should containers run as non-root?

A. Defense in depth. If the app is compromised, the attacker doesn't have root inside the container — limiting blast radius and making escapes harder. Always add a `USER` directive.

Q. How do you scan images for vulnerabilities?

A. Trivy in CI: `trivy image --severity HIGH,CRITICAL --exit-code 1`. Or Docker Scout, Snyk, Grype. Fail the build on critical CVEs. Rescan periodically because new CVEs are disclosed against existing images.

Q. What happens when a container exceeds its memory limit?

A. Linux OOM killer terminates a process inside (usually PID 1, killing the container). `State.OOMKilled` becomes true. If a restart policy is set, it restarts — masking but not fixing the underlying problem (which is usually a memory leak).

Q. How do you reduce a Docker image's size?

A. (1) Use a minimal base (alpine, distroless, scratch). (2) Multi-stage build — leave compilers/dev deps in builder stage. (3) `.dockerignore` to keep junk out. (4) Combine `RUN` steps; clean caches in the same layer. (5) `docker history` to see which layer is the offender.

Q. What's the difference between Alpine and Debian-based images?

A. Alpine uses `musl libc` and a tiny base — ~5MB. Debian uses `glibc` — ~75MB. Alpine is smaller and has fewer CVEs but can have compatibility issues with `glibc`-only binaries (some Node native modules, older Python wheels). Default to Alpine; fallback to Debian-slim if you hit `musl` issues.

Q. Describe a production-grade CI/CD pipeline for a containerized service.

A. On commit: build multi-stage image, run tests inside, scan with Trivy (fail on critical), push to ECR with `git-sha` tag, render new task definition with the new image, update ECS service, wait for stability, rolling deploy gated by health checks, automatic rollback on failure. Secrets from Secrets Manager via IAM task role. Total wall time ~5–10 minutes.

Q. What's Amazon ECR and what role does it play?

A. AWS-managed private Docker registry. IAM-authenticated, integrated with ECS/EKS/Lambda. Hosts your built images. Supports image scanning, lifecycle policies (auto-cleanup of old images), and OCI image format.

Q. What's an ECS task definition and how is it different from a Compose service?

A. A versioned JSON spec for one or more containers — image, CPU/memory, env, ports, IAM role, log driver. Conceptually similar to a Compose service but specifically for ECS. Each revision is immutable; deploys reference a specific revision.

Q. Difference between Fargate and ECS on EC2?

A. Fargate — serverless: AWS manages the infra; you pay per task vCPU-hour and GB-hour. EC2 mode — you manage the instances; you pay for instances. Fargate is the right default for most teams; EC2 is for specific use cases (GPU workloads, sustained high utilization where reserved EC2 is cheaper).

Q. How do you do a zero-downtime deploy in ECS?

A. Configure the service with healthy/maximum percent (e.g., 100% / 200%). Define container and target-group health checks. New tasks start, become healthy, register with the load balancer, old tasks deregister and drain — all in a rolling fashion. Automatic rollback if new tasks fail health checks.

Q. Why use OIDC instead of long-lived AWS access keys in GitHub Actions?

A. No persistent secrets in GitHub. Each workflow run gets a short-lived OIDC token, exchanged for time-limited STS credentials via an IAM role with a trust policy scoped to a specific repo/branch. Less to leak, easier to audit, no rotation needed.

Q. A container crashes in production. Walk me through your debug.

A. (1) Get the logs (CloudWatch / `docker logs`). (2) Inspect: exit code, OOMKilled, stopped reason. (3) Check resource graphs around the crash time — CPU/memory spikes? (4) Reproduce locally: pull the exact image SHA, override entrypoint to a shell, manually invoke the command. (5) Compare env: prod-only vars or missing IAM permissions? (6) Once the cause is known, ship the fix and add an alert so this class of failure surfaces sooner next time.

A.3 · 30-day study plan to job-ready

You said 4–6 weeks. Here is the 30-day version. Compress only if you actually do everything.

Week 1 — Networking & storage (Level 3)

Day	What to do	Output / proof of work
1	Read 3.0 – 3.3 (mental model, networking drivers, container DNS, port mapping). Run every command shown.	Can explain default bridge vs user-defined bridge from memory.
2	Read 3.4 – 3.6 (volumes, bind mounts, tmpfs, env & secrets). Lab 3.1 (DNS test) + Lab 3.3 (live reload bind mount).	Two networks tested, both labs working.
3	Lab 3.2 (Flask + Postgres milestone). Get it fully working with persistent volume.	Stack works end-to-end. Data survives container delete.
4	Lab 3.4 (volume backup & restore) + Lab 3.5 (multi-network isolation).	Both labs passing.
5	Lab 3.6 (debug broken connection). Pitfalls section — review each.	Can debug a "containers can't reach each other" scenario aloud.
6	Q&A — basic 15. Cover answer, say aloud, check.	15/15 right on first pass.
7	Q&A — advanced 10 + scenario 10.	Stumble on ≤ 2 questions.

Week 2 — Docker Compose (Level 4)

Day	What to do	Output / proof of work
8	Read 4.0 – 4.3 (Compose mental model, file structure, all service keys).	Can write a 2-service compose.yml from scratch in 5 minutes.
9	Read 4.4 – 4.7 (CLI, networks, healthchecks, env files). Lab 4.1.	Comfortable with up/down/exec/run.
10–11	Lab 4.2 — the 3-tier stack milestone. Take two days. Get it perfect.	Frontend + API + DB + Cache, two networks, healthchecks, persistent volume, env file. Demo-worthy.
12	Read 4.8 – 4.9. Lab 4.3 (dev override) + Lab 4.4 (prod override).	Same stack, two configs, one command difference.
13	Lab 4.5 (healthcheck conditional start) + Lab 4.6 (scaling with reverse proxy).	Both working. Watched start order in logs.
14	Q&A all 35. Then take a real compose file from a GitHub repo you don't know and explain it line-by-line aloud.	Can read any Compose file confidently.

Week 3 — Production hardening + CI (Level 5 first half)

Day	What to do	Output
15	Read 5.0 – 5.2. Lab 5.1 (harden Dockerfile).	Old vs new image size + non-root verified.
16	Read 5.3 – 5.4. Lab 5.2 (Trivy scan).	Scanned at least 3 images; understand CVE severity.
17	Read 5.5 (GitHub Actions). Lab 5.3 (build & push to GHCR).	Working CI workflow; image visible in your repo's packages.
18	Add Trivy step to the workflow. Introduce a deliberate vulnerability; watch CI fail. Fix it.	CI catches a real CVE.
19	Multi-arch build (linux/amd64 + arm64). Confirm both digests in the manifest.	Multi-arch image working.
20	Buffer day. Review week 3. Re-do anything shaky.	Notes on weak spots.
21	Q&A 5.10 basic 15 + advanced 10.	Confident on theory.

Week 4 — AWS deploys (Level 5 second half) + capstone

Day	What to do	Output
22	Read 5.6 (ECR). Lab 5.4 (push to ECR manually, then from GitHub Actions with OIDC).	Image in ECR. No long-lived AWS keys in GitHub.
23	Read 5.7 (ECS). Skim AWS console: clusters, task defs, services. Watch one tutorial video on Fargate.	Mental model locked in.
24–25	Lab 5.5 (deploy to Fargate). Take two days. Get the container reachable on a public IP.	curl your AWS container; CloudWatch logs flowing.
26–27	Lab 5.6 (end-to-end pipeline capstone). Two days for this — get every step working. Demo it to yourself.	git push → 5 min later, change is live in AWS. Rollback works.
28	Pitfalls section + scenario Q&A. Practice the "describe your pipeline" question 5 times aloud.	Can deliver the full pipeline narrative without notes.
29	Mock interview with yourself: top-30 Q&A in appendix. Time it.	≥27/30 right; under 60 sec each.
30	Put Lab 5.6 on your GitHub. Write a README explaining the pipeline. Add Lab 4.2 too. Link these on your resume.	Two public, demonstrable projects.



This plan will fail you if: you read without doing labs, you skip the Q&A spoken-aloud practice, you don't put the AWS capstone on your CV, or you don't budget AWS Free Tier (\$5–10 across Week 4 if you tear down — you'll need a working AWS account).

A.4 · What's beyond this PDF

This document gets you job-ready on Docker. To clear interview rounds at product companies, you still need:

Phase 6: Kubernetes (next in your blueprint)

Kubernetes is Docker + orchestration at scale. Every product company runs on it. Your existing K8s roadmap covers this — start there once Docker Level 5 feels effortless. Concretely: Pods, Deployments, Services, ConfigMaps/Secrets, Ingress, Helm, eventually CKA certification.

Phase 7: Terraform / Infrastructure as Code

Everything in Level 5 — ECR repos, ECS clusters, IAM roles — should ultimately be in Terraform, not clicked in console. Hashicorp's free intro course is the place to start. Pair with your existing AWS knowledge.

Observability — the universal skill

Containers without observability are a liability. Learn Prometheus + Grafana (metrics), and one of: Loki / ELK / Datadog (logs), Jaeger / X-Ray (tracing). Even basic CloudWatch fluency makes you valuable.

Things to read

- **Docker docs themselves** — docs.docker.com. Underrated. Read the *Concepts* section start-to-finish.
- **"Docker Deep Dive"** by Nigel Poulton — the textbook. ~300 pages, worth every one.
- **Docker Security best practices** by Snyk, Aqua — free PDFs, current threat landscape.
- **AWS ECS documentation** — better than most tutorials. Read the developer guide cover to cover.

Things to do, in priority order

1. **Build something real with this stack.** Pick one of your existing projects (CascadeHQ comes to mind) and containerize it properly — multi-stage Dockerfile, Compose for local dev, GitHub Actions to ECR, ECS Fargate for staging. *Real production experience beats every certification.*
2. **Contribute to one open-source project's Docker setup.** Even fixing a Dockerfile to add a non-root user is a real PR you can point to.
3. **Get one certification, eventually.** AWS Solutions Architect Associate first (broad), then CKA (narrow but deep). Skip the Docker certification — it's not respected by hiring managers.
4. **Mock interview practice.** Find a senior on LinkedIn willing to do a 30-minute mock. The feedback compounds.

The honest closing



This PDF is 60+ pages. Reading it takes 4-6 hours. Reading it doesn't make you a DevOps engineer. **The labs do.**

If at the end of 30 days you can't do Lab 5.6 from memory — the full GitHub → ECR → ECS pipeline — you are not job-ready yet, regardless of how much you've read.

If you can — you're more prepared than 80% of people calling themselves DevOps engineers in Hyderabad. Put it on your CV. Demo it in interviews. Walk in confident.

Direction over speed. You picked the right direction. Now move.

— End of Docker Mastery, Levels 3, 4, 5 —
Built for Vartha · Updated May 2026